RESEARCHLY AI: AI-POWERED RESEARCH ASSISTANT AND COLLABORATION PLATFORM FOR UNIVERSITY STUDENTS

RESEARCH INTEGRITY AND COMPLIANCE MODULE: CITATION, GAP ANALYSIS, PROPOSAL AND MIND-MAP SUPPORT

Individual Thesis

Kariyawasam K.D.T.

(IT22210524)

Group ID: R26-IT-116

Dissertation submitted in partial fulfilment of the requirements for the
Bachelor of Science (Hons) in Information Technology
Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

This thesis reports my individual component (Module 1: Research Integrity and Compliance Module) of group project R26-IT-116; the integrated solution is described in the common group report. Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Student name	Student ID	Signature	Date
Kariyawasam K.D.T.	IT22210524		

Supervisor certification. The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Sanjeevi Chandrasiri		
Co-supervisor: Ms. Ishara Weerathunga		

ABSTRACT

Final-year students must format references correctly, justify a research gap, draft a proposal and organise their ideas, yet each task is served by a separate tool and none uses the institution's own research as evidence. This thesis presents the Research Integrity and Compliance Module (Module 1) of Researchly AI, an integrated research assistant built for SLIIT. The module combines a deterministic citation parser and APA/IEEE formatter with CrossRef lookup; a research-gap analyser that retrieves author-stated limitations and future-work sentences from 553 gap statements mined from SLIIT papers and ranks them by similarity, type, recency and novelty; a proposal generator that composes a structured draft from the most similar of 3,858 SLIIT papers and lists them as sources; and a Gemini-assisted mind-map builder. A Design Science Research approach was followed, and the module was deployed as a FastAPI micro-service behind an authenticated API gateway with a Next.js interface.

On 3,866 SLIIT references the citation engine extracted the year in 100%, the title in 98.0% and author surnames in 90.9% of cases (all fields 89.1%) with a median latency of 0.077 ms, meeting the 90% field-level objective. Functional validation of 13 live test cases, signed off by the supervisor, produced 8 passes, 4 partial results and 1 failure (subsequently fixed). Proposal drafts were fully traceable to on-topic SLIIT papers, whereas gap analysis lacked a confidence gate for poorly covered topics. The work shows that a retrieval-first, rules-where-possible design can provide explainable integrity support on CPU-only infrastructure.

Keywords: citation parsing, research gap analysis, retrieval-grounded proposal generation, mind maps, Sentence-BERT

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Ms. Sanjeevi Chandrasiri, for her continuous guidance, critical feedback and encouragement throughout this research, and to my co-supervisor, Ms. Ishara Weerathunga, for her constructive advice on the scope, methodology and evaluation of my component. Their review of the validation evidence in August 2026 was instrumental in shaping the final version of this work.

I am grateful to the academic staff of the Department of Information Technology, Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT), for the learning environment, lecture sessions and resources provided through the IT4010 Research Project module. I also thank the authors of the SLIIT research papers whose openly available abstracts form the evidence base of this module.

A special thanks goes to my research group members, S P U Gunathilaka (IT22055194), N V Hewamanne (IT22135902) and H W S S Jayasundara (IT22553096), with whom the integrated Researchly AI platform was designed, built and deployed. I also acknowledge the open-source communities behind Sentence-Transformers, scikit-learn, statsmodels, XGBoost, FastAPI, Next.js and Supabase, whose tools made this work possible. This research received no external funding. Finally, I thank my family and friends for their patience and support.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION ..... **i**
ABSTRACT ..... **ii**
ACKNOWLEDGEMENTS ..... **iii**
TABLE OF CONTENTS ..... **iv**
LIST OF FIGURES ..... **iv**
LIST OF TABLES ..... **iv**
LIST OF ABBREVIATIONS ..... **iv**
LIST OF APPENDICES ..... **iv**
1 INTRODUCTION **1**
1.1 Background and Overall Project Context 1
1.2 Individual Component Context 2
1.3 Problem Statement 3
1.4 Research Gap 4
1.5 Aim and Objectives 4
1.6 Research Questions 5
1.7 Scope, Limitations and Assumptions 5
1.8 Individual Contribution 6
1.9 Thesis Structure 6
2 LITERATURE REVIEW AND RELATED WORK **7**
2.1 Review Method and Scope 7
2.2 Domain and Theoretical Foundations 7
2.2.1 Bibliographic reference parsing 7
2.2.2 Sentence embeddings and semantic retrieval 7
2.2.3 Research gaps and idea generation 8
2.2.4 Retrieval-augmented generation 8
2.2.5 Mind maps and concept graphs 8
2.3 Related Research 8
2.4 Existing Systems, Products and Techniques 9
2.5 Gap Synthesis and Conceptual Framework 9
3 RESEARCH METHODOLOGY **11**
3.1 Research Design 11
3.2 Data, Participants and Study Materials 11
3.3 Experimental Procedure 11
3.4 Measures and Evaluation Metrics 12
3.5 Data Analysis 12

3.6 Ethics, Privacy, Security and Responsible AI 12
3.7 Reproducibility and Research Artefacts 13
4 PRODUCT AND SOLUTION DESIGN 14
4.1 Requirements 14
4.2 Component Architecture within the Complete Solution 15
4.3 Design Alternatives and Decisions 15
4.4 Detailed Design 16
4.4.1 Citation parser and formatter 16
4.4.2 Research gap analysis 18
4.4.3 Proposal generator 19
4.4.4 Mind map builder 20
4.5 Standards and Professional Practice 20
5 IMPLEMENTATION, INTEGRATION AND TESTING 22
5.1 Development Environment and Technology Stack 22
5.2 Component Implementation 22
5.3 Integration with the Other Components 27
5.4 Verification and Software Testing 28
5.5 Deployment and Operation 29
6 RESULTS AND VALIDATION 31
6.1 Research Results 31
6.1.1 RQ1: Citation parsing accuracy 31
6.1.2 RQ2: Gap retrieval relevance 32
6.1.3 RQ3: Grounded proposal drafting 33
6.2 Product and Technical Results 33
6.3 User and Stakeholder Validation 34
6.4 Integration Results 34
6.5 Summary of Findings 34
7 DISCUSSION 36
7.1 Interpretation of Findings 36
7.2 Comparison with Literature Baselines and Existing Solutions 36
7.3 Research Contribution 36
7.4 Product Contribution 37
7.5 Limitations and Threats to Validity 37
7.6 Implications 37
8 COMMERCIALISATION AND SUSTAINABILITY 39
8.1 Value Proposition and Target Users 39
8.2 Market and Competitive Position 39
8.3 Business and Sustainability Model 39

8.4 Scalability, Intellectual Property and Compliance 40
8.5 Contribution to the Sustainable Development Goals 40
9 CONCLUSION AND FUTURE DIRECTIONS 41
9.1 Conclusion 41
9.2 Achievement of Objectives 41
9.3 Individual Research and Product Contributions 41
9.4 Future Directions 42
REFERENCES 43
GLOSSARY 46
APPENDICES 47
Appendix A: Extended Results and Test Evidence 47
Appendix B: Academic Integrity and AI-Use Disclosure 49
Appendix C: Additional Interface Screenshots 50
Appendix D: User Acceptance Testing Protocol 51

LIST OF FIGURES

Figure 1.1: High-level architecture of the Researchly AI platform showing the presentation, integration, intelligence and data layers 2
Figure 2.1: Student journey through the integrated platform; Module 1 features support gap analysis, proposal drafting and citation 10
Figure 4.1: Component architecture of the Research Integrity and Compliance Module 15
Figure 4.2: Citation parsing, lookup and formatting pipeline 17
Figure 4.3: Research gap analysis pipeline (retrieval layer and intelligence layer) 19
Figure 4.4: Retrieval-grounded proposal generation with fallback tiers 20
Figure 5.1: Citation Parser page: an IEEE-shaped conference reference parsed as a conference paper with 75% confidence and rendered in APA 24
Figure 5.2: Similar SLIIT papers retrieved by SBERT for the parsed citation, with match scores and RDA links 24
Figure 5.3: Gap analysis input: topic query mode with sample topics and year filters 25
Figure 5.4: Ranked research gaps for "machine learning for crop disease detection in Sri Lanka", each grounded in a cited SLIIT paper 26
Figure 5.5: Generated proposal sections — problem statement, objectives, methodology and expected outcomes — for an IoT smart-irrigation topic 26
Figure 5.6: Source SLIIT papers used to ground the generated proposal, with similarity scores 27
Figure 5.7: Request flow from a Next.js page through the gateway to a FastAPI service, including the fallback decision 28
Figure 5.8: Continuous deployment pipeline of the platform 30
Figure 6.1: Citation parser field-level accuracy on 3,866 SLIIT references 31
Figure 6.2: Composition of the gap index by gap type 33
Figure 6.3: Validation outcomes per module (Module 1 in the top row) 35
Figure A.1: Mind Map Builder output for a CNN plant-disease abstract, produced by the local fallback when Gemini JSON parsing failed (validation case M1-15) 47
Figure A.2: Tiered concept summary (central, primary, secondary, detail) under the mind-map graph 48
Figure C.1: Cross-domain opportunities produced by the gap intelligence layer 50
Figure C.2: Proposal generator input form with topic and domain 50

LIST OF TABLES

Table 1.1: Research gaps addressed by Module 1 4
Table 2.1: Related research informing Module 1 9
Table 2.2: Comparison of existing products with Module 1 9
Table 3.1: Data and study materials used by Module 1 11
Table 3.2: Evaluation measures for Module 1 12
Table 4.1: Functional requirements of Module 1 14
Table 4.2: Non-functional requirements of Module 1 14
Table 4.3: Design decisions and rationale for Module 1 15
Table 5.1: Technology stack 22
Table 5.2: Key Module 1 implementation files 22
Table 5.3: Functional validation of Module 1 (28 August 2026) 29
Table 6.1: Citation parser benchmark (n = 3,866) 31
Table 6.2: Feature completion status of Module 1 33
Table 8.1: Indicative revenue model (from the approved proposal) 39
Table 9.1: Achievement of objectives 41
Table A.1: Citation engine behaviour on six reference shapes (final version) 47
Table B.1: AI-use disclosure 49
Table D.1: UAT tasks and recording sheet for Module 1 51

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
APA	American Psychological Association
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DOI	Digital Object Identifier
DSR	Design Science Research
F1	Harmonic mean of precision and recall
FR	Functional Requirement
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LoRA	Low-Rank Adaptation
ML	Machine Learning
NER	Named Entity Recognition
NFR	Non-Functional Requirement
NLP	Natural Language Processing
PDF	Portable Document Format
PDPA	Personal Data Protection Act (Sri Lanka, No. 9 of 2022)
RAG	Retrieval-Augmented Generation
RDA	Research and Digital Archive (SLIIT institutional repository)
REST	Representational State Transfer
RLS	Row-Level Security
SBERT	Sentence-BERT
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
SO	Specific Objective
SUS	System Usability Scale
TF-IDF	Term Frequency–Inverse Document Frequency

Abbreviation	Description
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator

LIST OF APPENDICES

Appendix A: Extended Results and Test Evidence.....47
Appendix B: Academic Integrity and AI-Use Disclosure.....49
Appendix C: Additional Interface Screenshots.....50
Appendix D: User Acceptance Testing Protocol.....51

1 INTRODUCTION

1.1 Background and Overall Project Context

University students face a research workflow that has grown far faster than the tools supporting it. Global scholarly output rose from roughly 2.1 million articles in 2010 to more than 3.3 million in 2022 [1], and a final-year undergraduate must survey that literature, select a defensible topic, find a suitable supervisor, cite sources correctly, avoid overlap with earlier work and write a structured proposal, usually within a single academic year and with limited supervision time. Each of these tasks is served by a separate tool — a reference manager, a search engine, a similarity checker, a spreadsheet of supervisors — and none of these tools share data [2], [3].

The group project R26-IT-116, **Researchly AI — an AI-powered research assistant and collaboration platform for university students**, addresses this fragmentation by placing four intelligent modules behind one web application and one shared data substrate: Module 1 (Research Integrity and Compliance), Module 2 (Collaboration and Recommendation), Module 3 (Research Data Collection and Management) and Module 4 (Research Performance Analytics and Visualization). All four modules are grounded in a corpus of 4,219 records drawn from SLIIT's institutional research repository (RDA), of which 3,858 carry usable abstracts, so that every recommendation, gap, forecast or similarity score refers back to real prior SLIIT research rather than to generic web content. The platform is deployed as a Next.js front end on Vercel, an Express.js API gateway and four FastAPI micro-services, backed by Supabase PostgreSQL with the pgvector extension (Figure 1.1).

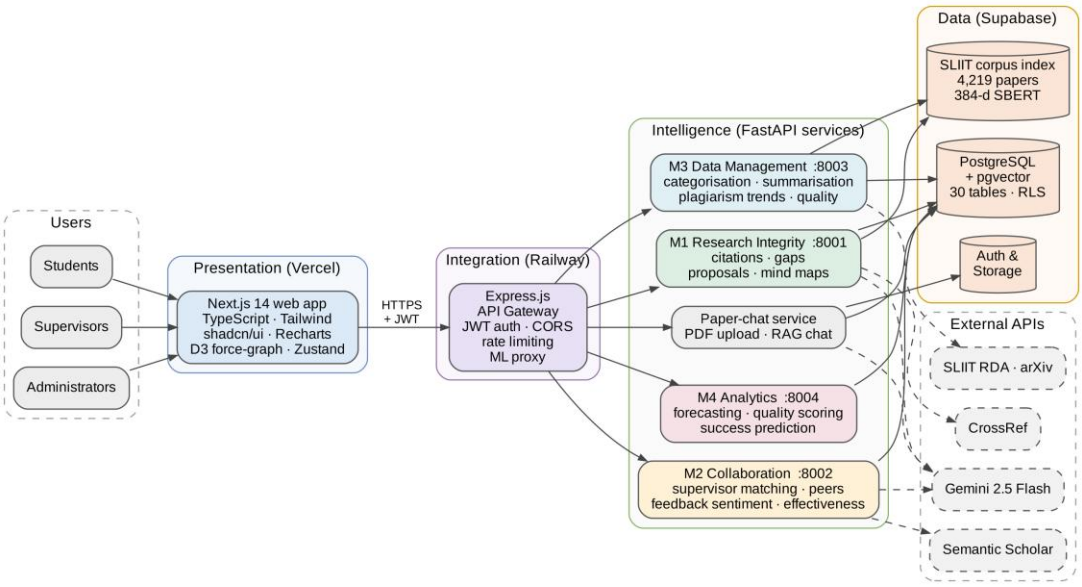


Figure 1.1: High-level architecture of the Researchly AI platform showing the presentation, integration, intelligence and data layers

This thesis reports the author's individual component, **Module 1: Research Integrity and Compliance Module**. The common group report describes the integrated solution, the shared infrastructure and the end-to-end evaluation; this document concentrates on the research problem, design, implementation and validation of the individual component, and on how it exchanges data with the other three modules.

1.2 Individual Component Context

Module 1, the **Research Integrity and Compliance Module**, is the part of the platform a student uses while *writing*. It turns a raw idea into a defensible, correctly cited research proposal through four connected features, each exposed as a page under `/citations` in the web application and as a route of the FastAPI service `services/module1-integrity` (port 8001):

- **Citation Parser and Formatter** — converts a pasted reference string of any rough shape into structured bibliographic fields and renders it in APA 7th edition [4] and IEEE [5] style, with in-text forms, missing-field warnings, CrossRef DOI/title lookup [6] and a reference-list builder.
- **Research Gap Analysis** — given a topic, a pasted paper or an uploaded PDF, retrieves evidenced research gaps from a semantic index of gap statements

mined from SLIIT papers and enriches them with gap type, year-wise trend, cross-domain opportunities, saturation warnings and suggested research directions.

- **Proposal Generator** — drafts a problem statement, objectives, methodology and expected outcomes that are explicitly grounded in the most similar prior SLIIT research, with the source papers listed for verification.
- **Mind Map Builder** — extracts the key concepts of an abstract or problem statement and renders them as an interactive, tiered concept graph for brainstorming.

The module consumes the shared SLIIT corpus and the fine-tuned Sentence-BERT (SBERT) encoder maintained with Module 3, and its outputs (topics, gaps, drafts) are the natural inputs to supervisor matching in Module 2 and to quality scoring and success prediction in Module 4.

1.3 Problem Statement

Undergraduate researchers at SLIIT, as elsewhere, must (i) reference sources correctly in the style mandated by their faculty, (ii) justify their topic with a research gap that is genuinely open, (iii) write a structured proposal and (iv) organise the concepts of their research area. These tasks are performed with disconnected tools: reference managers such as Zotero require metadata to be entered or imported per source [7]; literature search engines return ranked papers but do not state what is *missing*; and proposal writing is unsupported altogether. Systems that extract bibliographic metadata, such as CERMINE [8], GROBID [9] and ParsCit [10], stop at structured fields and do not render a submission-ready citation, while recent LLM-based idea generators produce plausible but unverifiable suggestions [11], [12].

The problem addressed by this component is therefore: **students lack a single, explainable tool that converts their raw references, topic and draft text into correctly formatted citations, evidence-backed research gaps, a grounded proposal draft and a concept map, using the institution's own body of prior research as evidence.**

1.4 Research Gap

The literature and product review in Chapter 2 identifies four specific gaps that the component targets (summarised in Table 1.1).

Table 1.1: Research gaps addressed by Module 1

ID	Gap	Evidence
G1	Citation tools either require manual metadata entry or extract metadata without rendering formatted APA/IEEE output from raw strings.	Reference managers [7]; extractors [8], [9], [10]
G2	Gap identification tools generate ideas rather than locating gaps that are stated, with evidence, in an institution's own literature.	LLM idea generation [11], [12]; citation-context analysis [13], [14]
G3	Proposal assistants rely on free generation and do not cite the prior work they draw on, so outputs cannot be verified.	Hallucination survey [15]; RAG [16]
G4	Mind-mapping tools are standalone and not connected to citation, gap or proposal workflows.	[17], [18]

The novelty of the component lies less in any single algorithm than in the combination: a deterministic, auditable citation engine; a retrieval-first design in which every gap and proposal fragment is traceable to a named SLIIT paper; and a single workflow that a student can complete in one sitting.

1.5 Aim and Objectives

Aim. To design, implement and evaluate an NLP-powered Research Integrity and Compliance Module that automates citation formatting, identifies evidenced research gaps, drafts grounded research proposals and generates concept mind maps for university students, as one integrated component of the Researchly AI platform.

Specific objectives. The objectives are carried over from the approved proposal and are each mapped to evidence in Chapter 6:

- **SO1** — Design and implement a citation parser that extracts fields from raw references and formats them in APA and IEEE styles, with a target field-level accuracy of at least 90%.
- **SO2** — Develop a research-gap identification algorithm based on semantic similarity over literature that returns at least three to five evidenced, ranked gaps for a given topic.

- **SO3** — Build a proposal assistant that generates a structured draft (problem statement, objectives, methodology, expected outcomes) grounded in retrieved prior research.
- **SO4** — Develop a mind-map generator that represents the concept structure of a text as an interactive graph.
- **SO5** — Integrate all features into the platform through the API gateway and shared data layer.

1.6 Research Questions

- **RQ1:** How accurately can a deterministic, rule-based engine parse and reformat real-world reference strings into APA and IEEE styles, and where does it fail?
- **RQ2:** Can sentence-embedding retrieval over gap statements extracted from an institutional corpus surface research gaps that are relevant to a student's topic, and how does performance change for poorly covered topics?
- **RQ3:** Does a retrieval-and-composition approach to proposal drafting produce drafts whose every claim is traceable to prior work, at an acceptable latency on CPU-only infrastructure?

1.7 Scope, Limitations and Assumptions

In scope: raw reference strings and DOIs; APA 7 and IEEE styles; English-language topics, pasted papers and PDF uploads; the SLIIT RDA corpus as evidence base; text-based mind maps. **Out of scope:** BibTeX/RIS import and export, citation styles other than APA and IEEE, and automated writing of complete proposals.

Assumptions: the shared corpus is representative of SLIIT research areas; the user reviews and edits every generated draft; gap statements written by authors in their own papers (limitations and future-work sentences) are a valid proxy for open research gaps.

1.8 Individual Contribution

The author designed and implemented the complete Module 1 stack: the FastAPI service (`app/routers/citation.py`, `gap_analysis.py`, `proposal.py`, `mindmap.py`), the service layer (`citation_engine.py`, `crossref_lookup.py`, `gap_analyzer.py`, `gap_intelligence.py`, `proposal_retriever.py`, `pdf_extractor.py`, `sliit_examples.py`), the training scripts that build the gap and proposal indices (`training/prepare_gap_corpus.py`, `train_gap_analyzer.py`, `prepare_proposal_corpus.py`, `train_proposal_retriever.py`), the four Next.js pages under `apps/web/src/app/(dashboard)/citations/`, the gateway route file `module1.routes.ts`, the spaCy citation-NER training experiment and the module validation report. As the team's integrator, the author also set up the monorepo, the API gateway, the CI/CD workflows and the Vercel/Railway deployment used by all four modules.

1.9 Thesis Structure

Chapter 2 reviews related literature and existing tools. Chapter 3 presents the research methodology. Chapter 4 describes requirements and design, and Chapter 5 the implementation, integration and testing. Chapter 6 reports results and validation evidence, Chapter 7 discusses them, Chapter 8 covers commercialisation and sustainability, and Chapter 9 concludes. The thesis follows the SLIIT documentation guidelines [19].

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

A structured search was carried out in IEEE Xplore, the ACM Digital Library, Google Scholar and Semantic Scholar for 2015–2026 publications, supplemented by foundational works. Search strings combined *citation parsing*, *reference string extraction*, *bibliographic metadata*, *research gap identification*, *scientific idea generation*, *retrieval-augmented generation*, *sentence embeddings* and *mind map generation*. Sources were retained if they proposed or evaluated a technique used, or rejected, in one of the four features; vendor documentation was used only for describing existing products.

2.2 Domain and Theoretical Foundations

2.2.1 Bibliographic reference parsing

Reference parsing is a structured sequence-labelling problem: a string must be segmented into author, title, venue, year, volume, issue, pages and identifiers. ParsCit used conditional random fields over token features [10]; CERMINE [8] and GROBID [9] combine heuristics with supervised models to extract metadata from whole PDFs and report high accuracy on titles and authors. Iqbal et al. note that a decade of NLP work on citations has concentrated on citation *context* and *function* rather than on producing correctly formatted references [13], and Jha et al. describe the same emphasis in scientometric pipelines [14]. Formatting rules themselves are fully specified by style manuals [4], [5], which suggests that, once fields are known, rendering is deterministic.

2.2.2 Sentence embeddings and semantic retrieval

BERT [20] produces contextual token representations but is inefficient for pairwise similarity at scale. Sentence-BERT fine-tunes BERT in a siamese configuration so that cosine similarity between pooled sentence vectors approximates semantic similarity, reducing search over 10,000 sentences from hours to seconds [21]. The distilled MiniLM family [22], and in particular `all-MiniLM-L6-v2` (384 dimensions), is a common CPU-friendly choice. Because vectors can be L2-normalised, cosine

similarity reduces to a dot product, so a whole index can be scored with one matrix–vector multiplication; for larger corpora, approximate methods such as HNSW [23], available in pgvector [24], are used.

2.2.3 Research gaps and idea generation

A research gap is a question that the literature acknowledges but has not answered. Authors routinely state such gaps in limitation and future-work sections, which makes them recoverable by text mining. Recent LLM agents such as ResearchAgent iteratively propose ideas over a literature graph [11]; Si et al. found that LLM-generated ideas were judged more novel but less feasible than expert ideas [12]. Both lines of work emphasise *generation*. For a student who must defend a gap before a panel, *evidence* — a citable paper that names the gap — matters more than novelty.

2.2.4 Retrieval-augmented generation

Retrieval-augmented generation conditions a generator on documents retrieved for the query, improving factuality on knowledge-intensive tasks [16]. Parameter-efficient fine-tuning such as LoRA [25] makes domain adaptation of large models feasible, but still requires GPU inference. Ji et al. survey hallucination in natural-language generation and show that even grounded generators can produce unsupported statements [15], a serious risk for an academic-integrity tool.

2.2.5 Mind maps and concept graphs

Mind maps and concept maps externalise the hierarchical structure of ideas and are associated with improved organisation of knowledge [26], [27]. Wei et al. generate mind maps automatically from document semantic structure [17], and Wen et al. show that knowledge-graph prompting improves LLM reasoning [18]. Large language models such as Gemini [28] can extract concepts and relations in one call, at the cost of non-determinism.

2.3 Related Research

Table 2.1 summarises the studies that most directly informed the design decisions of this component.

Table 2.1: Related research informing Module 1

Study	Approach	Relevance and limitation
Councill et al. (ParsCit) [10]	CRF reference-string parser	Field extraction only; no style rendering; requires training data.
Tkaczyk et al. (CERMINE) [8]	Rules + ML over full PDFs	Strong metadata extraction; does not format citations in a target style.
Lopez (GROBID) [9]	Cascaded CRF / deep models	Industrial-strength; heavy deployment; output is TEI XML, not APA/IEEE text.
Iqbal et al. [13]	Survey of in-text citation analysis	Confirms focus on context/function, motivating G1.
Baek et al. (ResearchAgent) [11]	LLM agent for idea generation	Generates ideas; no explicit evidence for gaps.
Si et al. [12]	Human study of LLM ideas	Ideas novel but less feasible; motivates grounding.
Lewis et al. (RAG) [16]	Retrieval + generation	Improves factuality; still generative; GPU cost.
Reimers and Gurevych (SBERT) [21]	Siamese sentence embeddings	Efficient semantic similarity used throughout the module.
Wei et al. [17]	Automatic mind-map generation	Standalone visualisation; not linked to integrity workflow.

2.4 Existing Systems, Products and Techniques

Table 2.2 compares the tools students currently combine to perform the tasks covered by Module 1.

Table 2.2: Comparison of existing products with Module 1

Product	Citation formatting	Gap analysis	Proposal drafting	Mind map
Zotero / Mendeley	Yes (after import)	No	No	No
Google Scholar	Copy-paste citation	No	No	No
General LLM chat	Unreliable	Generated, unverified	Generated, unverified	Text only
Mind-mapping apps	No	No	No	Manual
Module 1	Raw string → APA/IEEE	Evidenced (SLIIT)	Grounded draft	Automatic, interactive

2.5 Gap Synthesis and Conceptual Framework

The review suggests a design principle for an integrity tool: **use deterministic methods where rules exist, retrieval where evidence exists, and generation only where interpretation is genuinely required.** Citation formatting is rule-governed;

gaps and proposal content can be retrieved from the institution's own literature; only concept extraction for mind maps benefits materially from an LLM. Figure 2.1 shows how this principle fits into the student journey supported by the integrated platform, with Module 1 features at the writing stages.

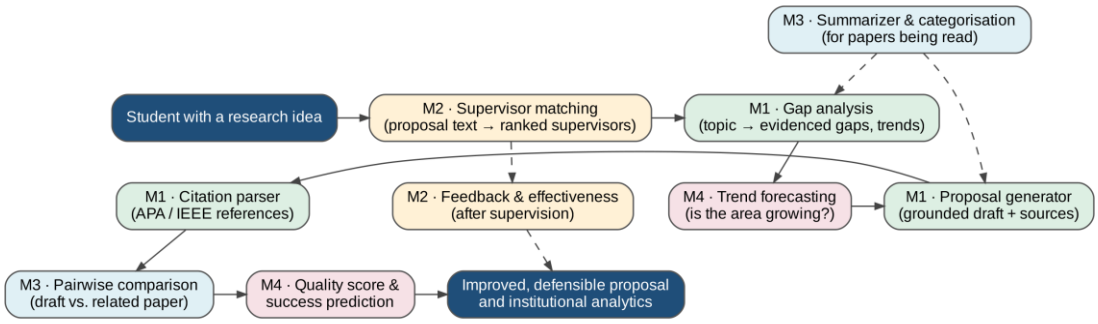


Figure 2.1: Student journey through the integrated platform; Module 1 features support gap analysis, proposal drafting and citation

3 RESEARCH METHODOLOGY

3.1 Research Design

The component follows Design Science Research (DSR), in which an artefact is designed to solve an identified problem and evaluated rigorously [29]. The six activities of the DSR methodology of Peffers et al. [30] were applied in two-week sprints: problem identification (Chapter 1), definition of objectives (Section 1.5), design and development (Chapters 4 and 5), demonstration (screenshots in Chapter 5 and Appendix C), evaluation (Chapter 6) and communication (this thesis and the group research paper).

3.2 Data, Participants and Study Materials

Table 3.1: Data and study materials used by Module 1

Material	Description	Use
SLIIT corpus	4,219 records from the SLIIT RDA repository; 3,858 with abstracts ≥ 50 characters; 1986–2026	Evidence base for proposals and similar papers
Gap corpus	553 gap sentences from 494 papers, labelled limitation (333), future work (117), scarcity (56), research gap (40), other (7)	Gap retrieval index
Proposal index	3,858 exemplars with SBERT embeddings (384-d), 13.3 MB pickle	Proposal generation and similar-paper search
Citation benchmark	3,866 reference strings generated from corpus metadata with gold authors, title and year	Evaluation of SO1
Style test set	Six hand-written references (APA journal, IEEE conference, IEEE journal, book, website, unstructured)	Qualitative robustness check
Validation scenarios	13 live API test cases executed on 28 August 2026 and signed off by the supervisor	Functional validation

No human participants were involved in model training. Supervisor review of the validation report constituted the expert evaluation; a student user-acceptance protocol is provided in Appendix D.

3.3 Experimental Procedure

1. Clean the corpus (shared with Module 3) and extract gap sentences using pattern families for limitations, future work and stated gaps (`prepare_gap_corpus.py`).

2. Encode gap sentences and paper records with the SLIIT-tuned SBERT encoder and persist the indices (`train_gap_analyzer.py`, `train_proposal_retriever.py`).
3. Implement and unit-check each feature against hand-constructed inputs.
4. Run the citation benchmark over 3,866 references and the six-style robustness set using the current version of `citation_engine.py`.
5. Execute the 13-case functional validation through the live FastAPI service, recording requests, responses and verdicts.
6. Fix defects found, re-test, and record remaining limitations.

3.4 Measures and Evaluation Metrics

Table 3.2: Evaluation measures for Module 1

Objective	Measure	Target
SO1	Field-level exact-match accuracy (year, title, author count, author surnames, all fields); parse latency	≥ 90% per field
SO2	Number of ranked gaps returned; topical relevance judged by evaluator; behaviour for out-of-corpus topics	3–5 relevant gaps
SO3	Relevance of retrieved exemplars; traceability of each section; latency	All sections traceable
SO4	Meaningful central node and concept hierarchy	Qualitative
SO5	End-to-end request through gateway with authentication	All routes reachable

3.5 Data Analysis

Benchmark results are reported as proportions with the evaluation size. Text comparisons normalise case and remove punctuation before matching; author surnames are compared in order. Functional validation results are classified as PASS, PARTIAL, FAIL or BLOCKED, with severity and root cause recorded for every non-pass. Because retrieval quality for gaps has no gold standard, relevance was judged qualitatively by inspecting each returned gap and its source paper.

3.6 Ethics, Privacy, Security and Responsible AI

- The corpus consists only of publicly available bibliographic records and abstracts from SLIIT's open research repository; full texts uploaded by users

are stored in a private Supabase Storage bucket protected by Row-Level Security [31].

- Personal data (names, e-mail addresses, research interests, ratings) are processed in line with the principles of Sri Lanka's Personal Data Protection Act No. 9 of 2022 — purpose limitation, data minimisation and access control [32].
- Every request to an intelligence service passes through the gateway, which verifies a Supabase-issued JSON Web Token [33] and applies rate limiting (600 requests per minute globally and 200 per minute for ML routes).
- Generative components (Gemini) are used only where interpretation is genuinely required and every retrieval-based output is traceable to a cited SLIIT paper, which reduces the risk of hallucinated content [15].
- Every generated proposal displays the source SLIIT papers so that students attribute ideas correctly; the interface states that drafts must be reviewed and rewritten by the student.

3.7 Reproducibility and Research Artefacts

All code, indices and evaluation scripts are in the project repository (`dilumonline2000/SLIIT-Research---Researchly-AI-project`). Index metadata (`models/trained_gap_analyzer/metadata.json`, `models/trained_proposal_retriever/metadata.json`) records corpus sizes and year distributions. The validation harness (`docs/module-reports/_test_module1.py`) and recorded results (`_results_module1.json`) allow every reported test to be re-run.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the proposal, stakeholder discussions with the supervisor and the gaps in Section 1.4. Table 4.1 lists functional requirements and Table 4.2 non-functional requirements.

Table 4.1: Functional requirements of Module 1

ID	Requirement
FR1	Parse a raw reference string into authors, title, year, venue, volume, issue, pages, DOI and URL, and detect source type.
FR2	Format a structured record in APA 7 and IEEE styles with in-text citations, and build a sorted (APA) or numbered (IEEE) reference list.
FR3	Look up metadata by DOI or title through CrossRef and route it through the same formatter.
FR4	Return similar SLIIT papers for a citation or topic.
FR5	Analyse a topic, pasted paper or PDF and return ranked, classified gaps with supporting papers, trends, cross-domain opportunities, saturation warnings and research directions; export an HTML report.
FR6	Generate a structured proposal grounded in retrieved exemplars, list source papers and export PDF.
FR7	Generate an interactive tiered mind map from text and export SVG/PNG.

Table 4.2: Non-functional requirements of Module 1

ID	Category	Requirement
NFR1	Performance	Citation parsing under 100 ms; retrieval features under 5 s on CPU after warm-up.
NFR2	Reliability	Every endpoint degrades gracefully (fallback tier) instead of returning an error to the UI.
NFR3	Explainability	Gaps and proposal content must cite a real SLIIT paper, author and year.
NFR4	Security	All routes require a valid Supabase JWT and are rate limited at the gateway.
NFR5	Usability	Responsive pages with sample inputs, copy buttons and downloadable reports.
NFR6	Maintainability	Feature logic isolated in service modules; indices rebuilt by scripts.

4.2 Component Architecture within the Complete Solution

Figure 4.1 shows the component. Each page calls the gateway with a JWT; the gateway route file applies `requireAuth` and `mlRateLimiter` and forwards the body to Module 1 through `callMlService(1, path, method, body)` with a 60-second timeout (PDF uploads are re-packaged as multipart form data). Inside the service, four routers delegate to a service layer. A three-tier fallback pattern applies throughout: a locally trained SBERT index is used first; Gemini 2.5 Flash is a second tier for proposal and mind-map generation; and a deterministic heuristic is the last resort, so that the product never hard-fails.

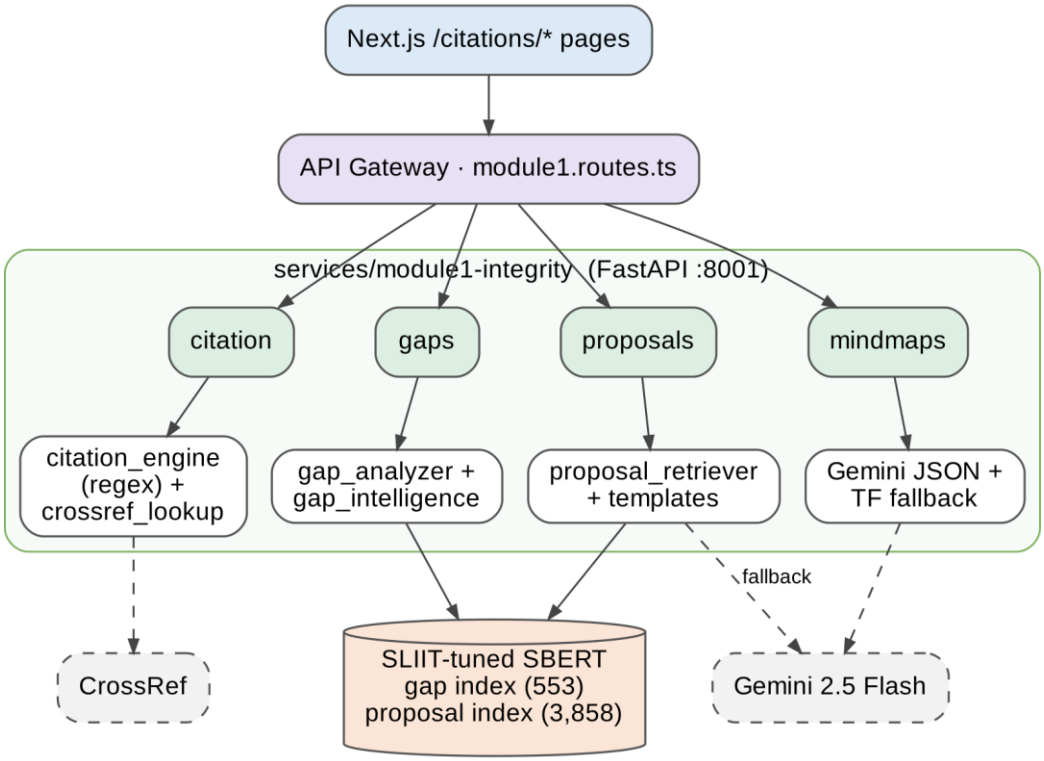


Figure 4.1: Component architecture of the Research Integrity and Compliance Module

4.3 Design Alternatives and Decisions

Table 4.3: Design decisions and rationale for Module 1

Concern	Alternative considered	Decision	Rationale
Citation parsing	Fine-tuned spaCy transformer NER (trained, best F1 0.995 on synthetic validation data)	Deterministic regex engine	Auditable, ~0.1 ms per reference, no GPU; style rules are fixed [4], [5]. The NER wrapper is retained for future use.

Concern	Alternative considered	Decision	Rationale
Proposal drafting	Mistral-7B / Llama-2 with LoRA and RAG [25], [16]	SBERT retrieval + composition, Gemini fallback	Zero hallucination: every sentence traceable; CPU-only Railway hosting.
Gap retrieval	BERTopic/LDA coverage analysis [34], [35]	Retrieval over stated gap sentences + rule-based intelligence layer	Gaps are evidenced by authors themselves; deterministic ranking.
Mind maps	KeyBERT + NetworkX [36]	Gemini concept extraction + term-frequency fallback	Hierarchy and importance need language understanding; best-effort feature.
Similarity search	FAISS / HNSW index	Exact dot product over normalised NumPy matrix	3,858 × 384 matrix scores in milliseconds; no extra dependency.

4.4 Detailed Design

4.4.1 Citation parser and formatter

Figure 4.2 shows the citation pipeline. `detect_source_type()` classifies the reference using regex hints: a URL without a DOI or journal hint indicates a website; "Proc.", "Conf." or "Workshop" a conference; "Journal", "Trans." or "Review" a journal; known publisher names a book; and a DOI with no other hint defaults to a journal. Dedicated patterns extract DOI (`10.\d{4,9}/…`), URL, publisher, volume, issue and page range. Every four-digit token between 1800 and 2099 is collected and the latest is taken as the publication year. If the string contains a quoted title — the IEEE shape — the quote is used as the anchor separating authors from title and venue; otherwise the string is split at the year marker (the APA shape). Authors are normalised to "Surname, Initials".

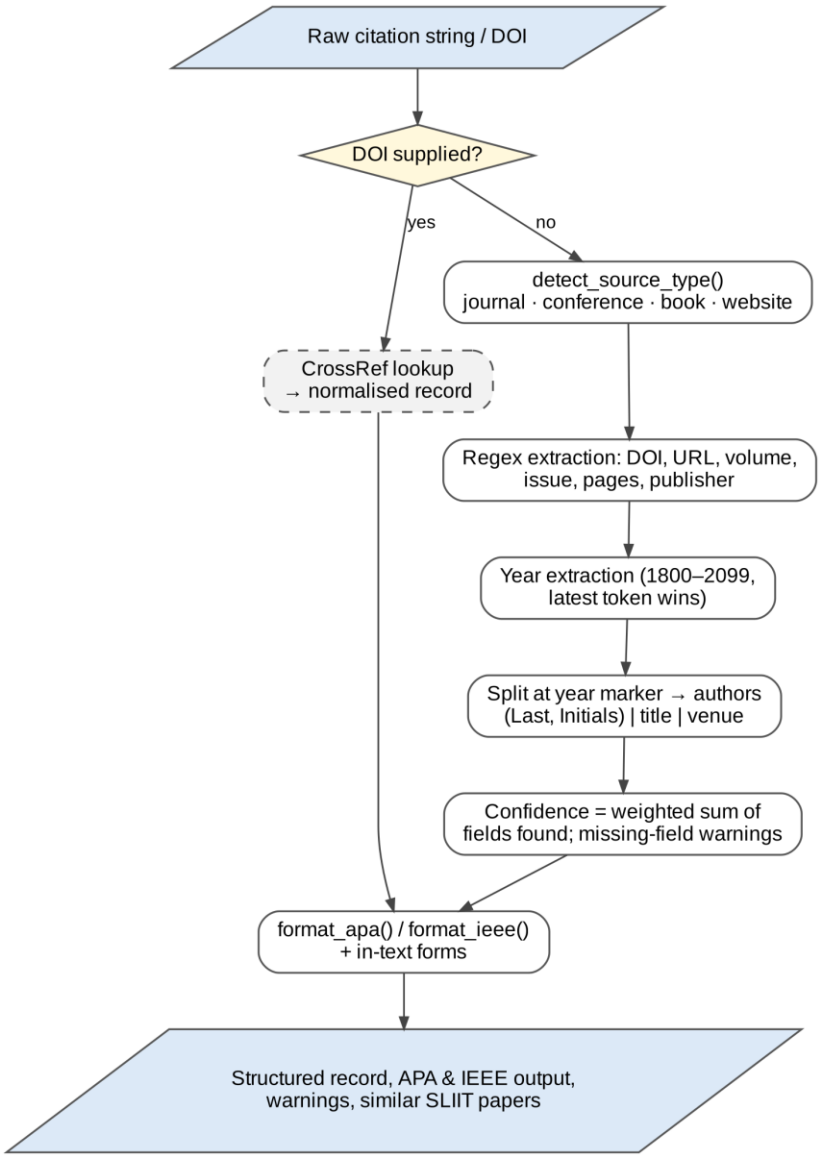


Figure 4.2: Citation parsing, lookup and formatting pipeline

A confidence score summarises how complete the parse is:

$$\text{confidence} = 0.25\cdot[\text{authors}] + 0.25\cdot[\text{title}] + 0.15\cdot[\text{year}] + 0.10\cdot[\text{venue}] + 0.10\cdot[\text{DOI}] \; (+ \; 0.05\cdot[\text{URL}] \text{ for websites})$$

`format_apa()` renders *Authors. (Year). Title. Journal, vol(issue), pages. DOI* with sentence-case titles (preserving acronyms and capitalising after a colon), while `format_ieee()` renders *Initials. Surname, "Title," Venue, vol., no., pp., Year, doi*. In-text forms follow the same duality: (Surname, Year), (Surname & Second, Year) or (Surname et al., Year) for APA and a bracketed running index for IEEE. The reference-

list builder preserves input order for IEEE and sorts alphabetically by first-author surname, then year, for APA. DOI and title lookups call the CrossRef REST API [6] and normalise its JSON into the same internal record, so that looked-up and typed citations share one formatter.

4.4.2 Research gap analysis

The gap pipeline (Figure 4.3) has a retrieval layer and an intelligence layer. The retrieval layer lazily loads the gap index — 553 gap sentences with metadata and 384-dimensional embeddings — and the SLIIT-tuned SBERT encoder (falling back to the base `all-MiniLM-L6-v2` model if fine-tuned weights are absent). The query is encoded and normalised, and similarity to every gap is computed as a single matrix–vector product. The top max(40, 5·k) candidates are scored on three axes:

$$\text{gap_score} = \min(1, \text{similarity} \times \text{type_weight}), \quad \text{type_weight} \in [0.95, 1.10]$$

$$\text{recency} = \min(0.95, 0.20 + (\text{current_year} - \text{paper_year}) / 12)$$

$$\text{novelty} = \min(1, 0.5 \cdot (1 - (\text{current_year} - \text{paper_year})/10) + 0.5 \cdot \text{similarity})$$

$$\text{multi_dim} = 0.45 \cdot \text{gap_score} + 0.20 \cdot (1 - \text{recency}) + 0.20 \cdot \text{novelty} + 0.15 \cdot \text{similarity}$$

Near-duplicate gaps are removed greedily: a gap is kept only if its cosine similarity to every gap already kept is below 0.78. The intelligence layer then classifies each gap into future-work, methodology, dataset, performance or domain categories using five regex pattern families; fits a least-squares slope to the yearly count of supporting papers over the last five years, labelling the topic rising (> +15% of mean per year), declining (< −15%) or stable; flags cross-domain opportunities among ten domains when fewer than 20% of the primary domain's papers also cover the second domain; flags saturated terms that appear in four or more matched paper titles; and synthesises up to five research directions with templated problem statements. For PDF or full-paper input, the text is split into abstract, methodology and conclusion and a focused query (1,500 + 1,000 + 800 characters) is built before the same pipeline is run.

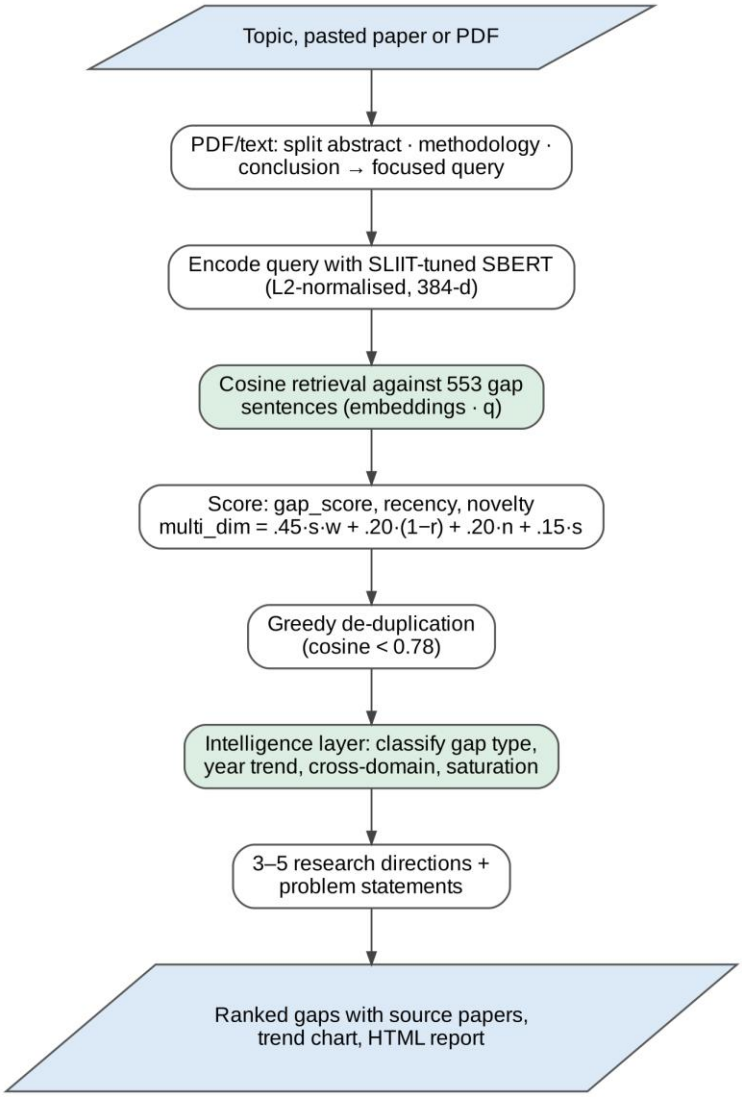


Figure 4.3: Research gap analysis pipeline (retrieval layer and intelligence layer)

4.4.3 Proposal generator

The proposal generator (Figure 4.4) embeds "topic (domain)" and retrieves the k most similar exemplars from the 3,858-paper proposal index. The problem statement quotes the top exemplar's problem statement or abstract (truncated to 350 characters) with attribution to its authors and year. Objectives start from two systematic-review objectives and add up to three action sentences extracted from exemplars with the pattern *propose* | *develop* | *design* | *evaluate* | *implement* | *analyse* | *assess* | *construct*, de-duplicated on an 80-character key. The methodology paragraph combines methodology snippets from up to three exemplars in a mixed-methods template, and

expected outcomes state the contribution and how many prior SLIIT studies are extended. If the index is unavailable, a Gemini prompt requests the same JSON schema; if that fails, a deterministic template is returned.

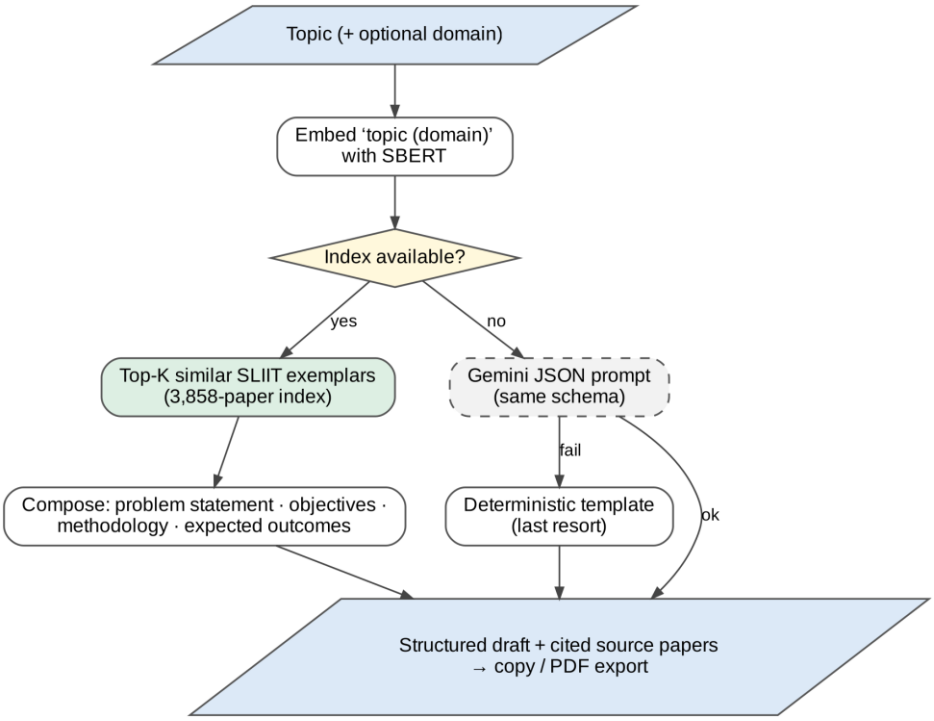


Figure 4.4: Retrieval-grounded proposal generation with fallback tiers

4.4.4 Mind map builder

A structured prompt asks Gemini for a JSON object containing a central concept and up to *max_nodes − 1* concepts with weights, a related-to-central flag and an optional parent, computed over the first 4,000 characters of the input. Each concept becomes a node, and edges connect to its parent or to the centre. On any Gemini failure, a local fallback counts words of four or more letters after removing about 35 stop-words; the most frequent term becomes the centre and weights are frequency ratios. In both paths, non-central nodes are sorted by weight and assigned tiers — top 25% primary, next 35% secondary, remainder detail — so the front end can colour nodes consistently.

4.5 Standards and Professional Practice

- Citation output follows the APA Publication Manual (7th ed.) [4] and the IEEE Reference Guide [5].

- Quality attributes were specified against ISO/IEC 25010 categories (performance efficiency, reliability, security, usability, maintainability) [37].
- Web security follows OWASP Top 10 guidance — authentication at the gateway, rate limiting, no secrets in source code (removed in commit history on 27 April 2026) [38].
- Code is typed (TypeScript, Pydantic models), modular and version-controlled with descriptive commit messages.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

Table 5.1 summarises the stack shared by all modules; the Module 1 service additionally depends on `sentence-transformers`, `pypdf`, `pdfplumber`, `httpx` (CrossRef) and `google-generativeai`.

Table 5.1: Technology stack

Layer	Technologies	Justification
Presentation	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, shadcn/ui, Recharts, D3 / react-force-graph-2d, Zustand, Axios	Server-side rendering, typed components, responsive layout and rich interactive charts in one framework [39]
Integration	Express.js API gateway (TypeScript), express-rate-limit, Supabase JWT verification, per-module proxy with 60 s / 150 s timeouts	Single entry point that authenticates once and isolates the front end from service locations [40]
Intelligence	Python 3.10+, FastAPI, sentence-transformers (all-MiniLM-L6-v2, fine-tuned), scikit-learn, XGBoost, statsmodels, PyMuPDF / pypdf / pdfplumber, Google Gemini 2.5 Flash client	Asynchronous typed micro-services; CPU-friendly models that fit the hosting budget [41], [42]
Data	Supabase PostgreSQL 15 + pgvector, Row-Level Security, Supabase Auth and Storage; pre-computed NumPy embedding matrices and pickled model bundles	Relational and vector search in one managed service, policy-based access control [24], [31]
DevOps	pnpm workspaces + Turborepo monorepo, Docker multi-stage images, GitHub Actions, Vercel, Railway, Azure Container Registry workflow, Playwright	Reproducible builds and automatic deployment on every push to main [43]

5.2 Component Implementation

Table 5.2: Key Module 1 implementation files

File	Lines	Responsibility
app/services/citation_engine.py	510	Source-type detection, field extraction, APA/IEEE formatting, in-text, reference list
app/services/gap_intelligence.py	592	Gap classification, scoring, trend, cross-domain, saturation, directions, HTML report
app/routers/gap_analysis.py	297	Topic, full-paper and PDF gap endpoints
app/routers/citation.py	241	Citation endpoints, confidence, CrossRef lookup, similar papers
app/services/proposal_retriever.py	229	Exemplar retrieval and section composition
app/services/gap_analyzer.py	215	Gap index loading, SBERT retrieval, de-duplication
app/routers/mindmap.py	166	Gemini concept extraction, fallback, tiers

File	Lines	Responsibility
training/prepare_gap_corpus.py	149	Gap sentence extraction from the corpus
web: citations/parser/page.tsx	821	Parser, DOI lookup and reference-list UI
web: citations/gaps/page.tsx	707	Gap analysis UI with trend chart and report download
web: citations/proposal/page.tsx	420	Proposal UI with PDF export (jsPDF)
gateway: routes/module1.routes.ts	209	Authenticated proxy routes incl. multipart PDF

The parser endpoint shows the general pattern: the router delegates to the engine and wraps its record with formatted outputs, warnings and confidence.

```
@router.post("/parse", response_model=ParseResponse)
async def parse(req: ParseRequest) -> ParseResponse:
    """Parse a raw citation string into structured fields + IEEE/APA
outputs."""
    from app.services import citation_engine
    parsed = citation_engine.parse(req.raw_text)
    return _to_response(parsed, source="regex")
```

Figures 5.1 to 5.5 show the implemented interface. The Citation Parser (Figure 5.1) offers three tabs (raw text, DOI lookup, reference list), sample inputs and a style switch, and displays the detected source type and confidence; Similar SLIIT papers are listed beneath (Figure 5.2).

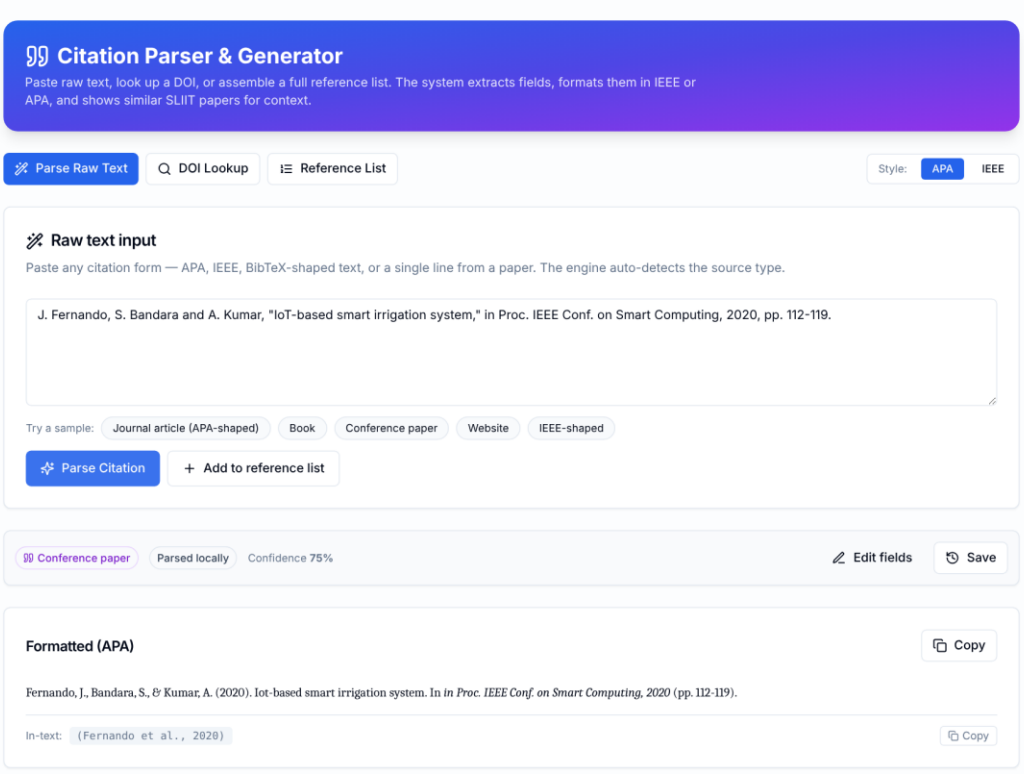


Figure 5.1: Citation Parser page: an IEEE-shaped conference reference parsed as a conference paper with 75% confidence and rendered in APA

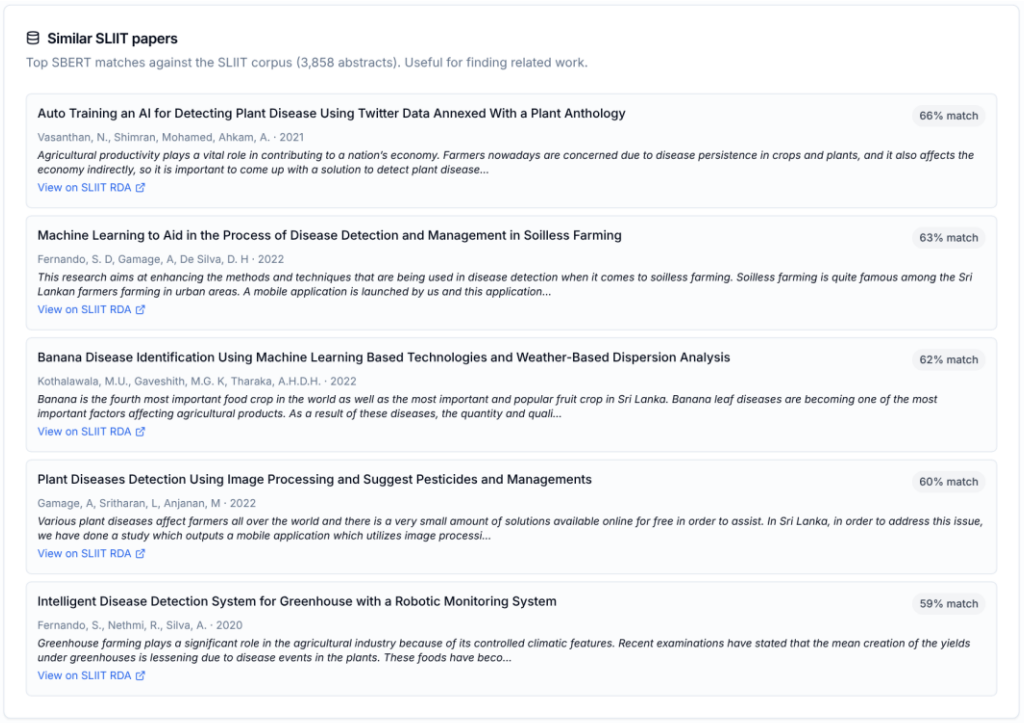


Figure 5.2: Similar SLIIT papers retrieved by SBERT for the parsed citation, with match scores and RDA links

The Research Intelligence page (Figure 5.3) accepts a topic, pasted text or a PDF with optional year filters and returns ranked gaps with similarity, novelty and recency and the source paper (Figure 5.4).

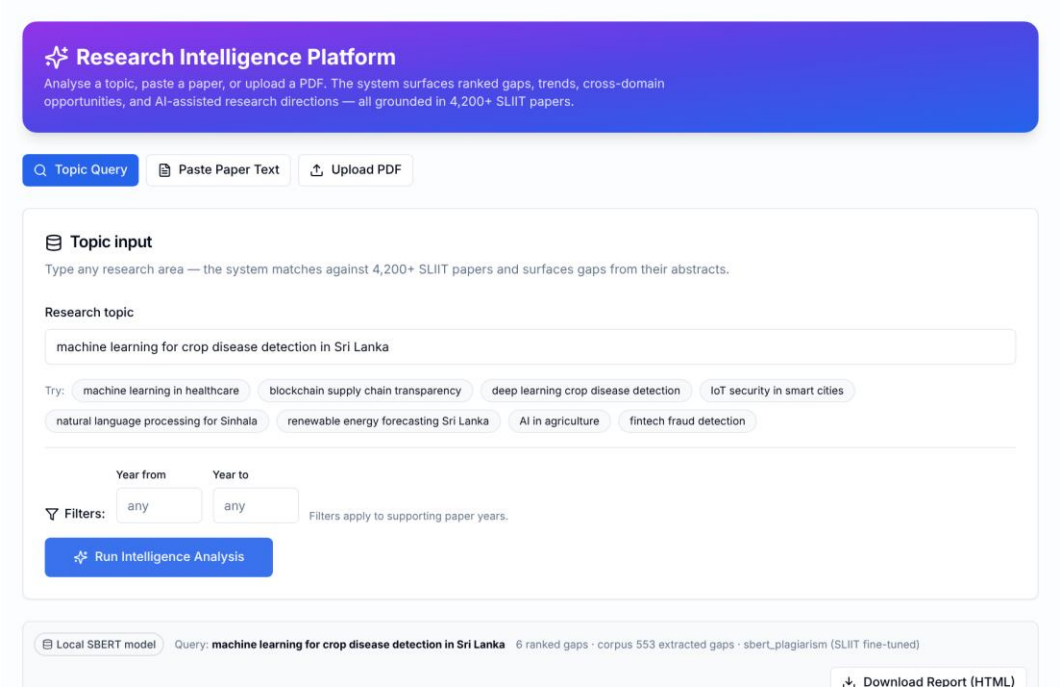


Figure 5.3: Gap analysis input: topic query mode with sample topics and year filters

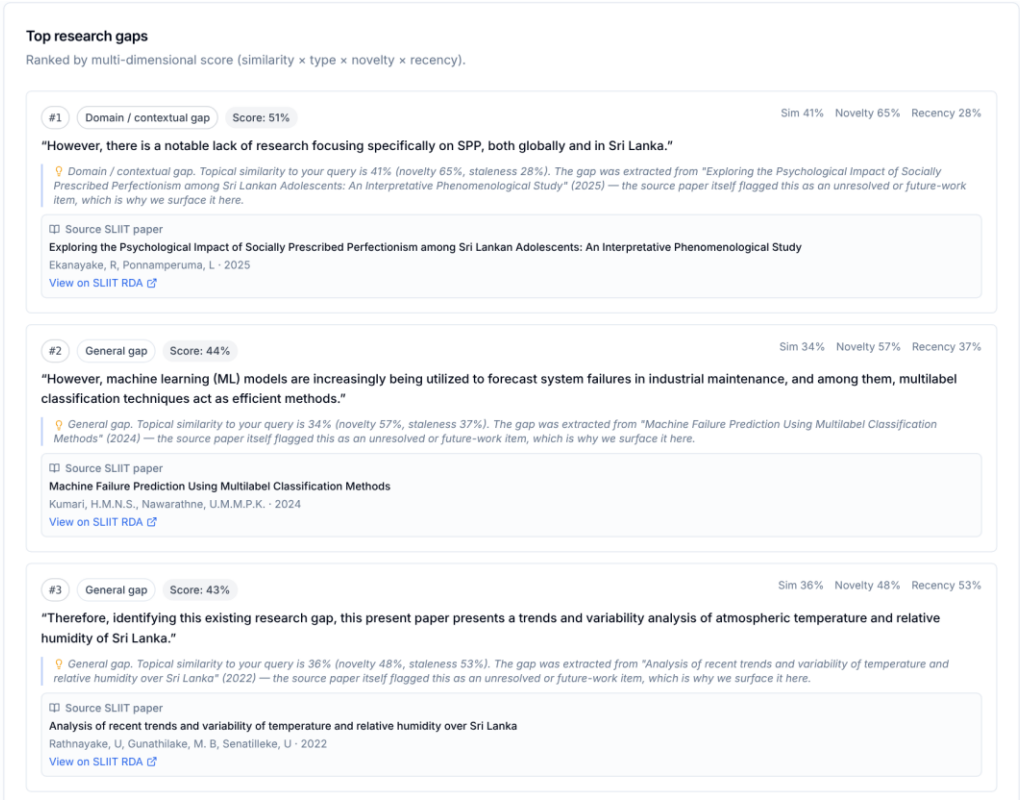


Figure 5.4: Ranked research gaps for "machine learning for crop disease detection in Sri Lanka", each grounded in a cited SLIIT paper

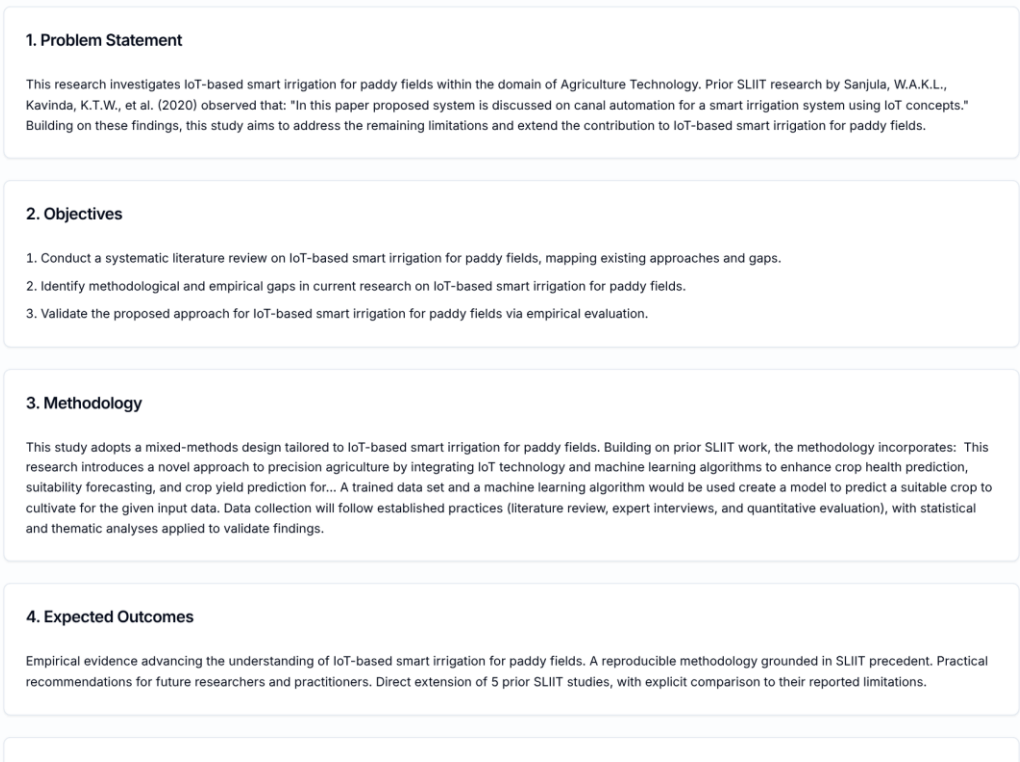


Figure 5.5: Generated proposal sections — problem statement, objectives, methodology and expected outcomes — for an IoT smart-irrigation topic

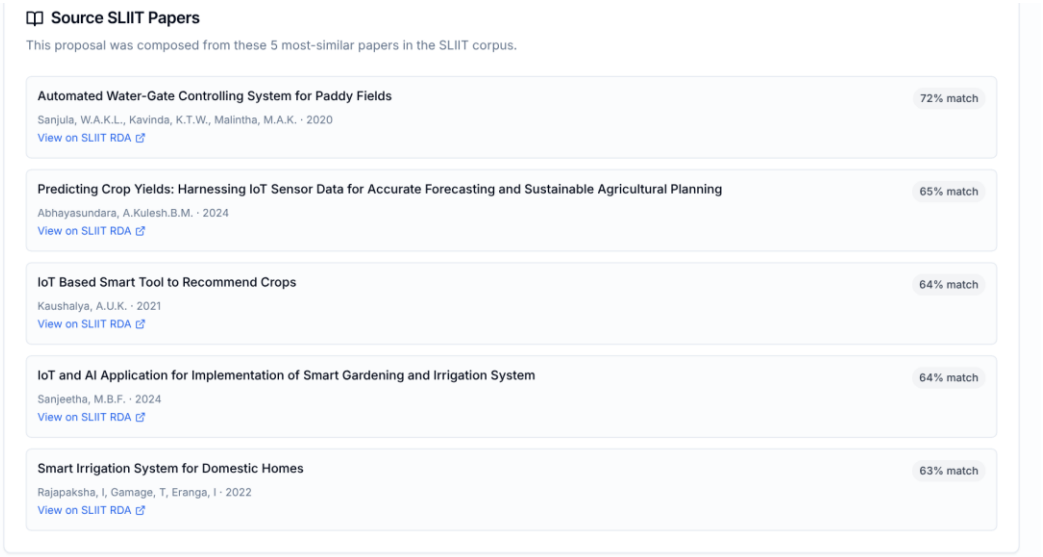


Figure 5.6: Source SLIIT papers used to ground the generated proposal, with similarity scores

5.3 Integration with the Other Components

- **Module 3 (data):** Module 1 reuses the SLIIT-fine-tuned SBERT model (`models/sbert_plagiarism`) and the corpus maintained by Module 3.
- **Module 2 (collaboration):** a topic refined through gap analysis can be pasted directly as the proposal text for supervisor matching, which uses the same embedding space.
- **Module 4 (analytics):** the generated proposal (title and abstract) can be scored for quality and success probability; trend forecasts complement the gap module's year-wise trend.
- **Platform:** all requests use the shared gateway, Supabase authentication and the common dashboard shell; Figure 5.7 shows the request flow common to every module.

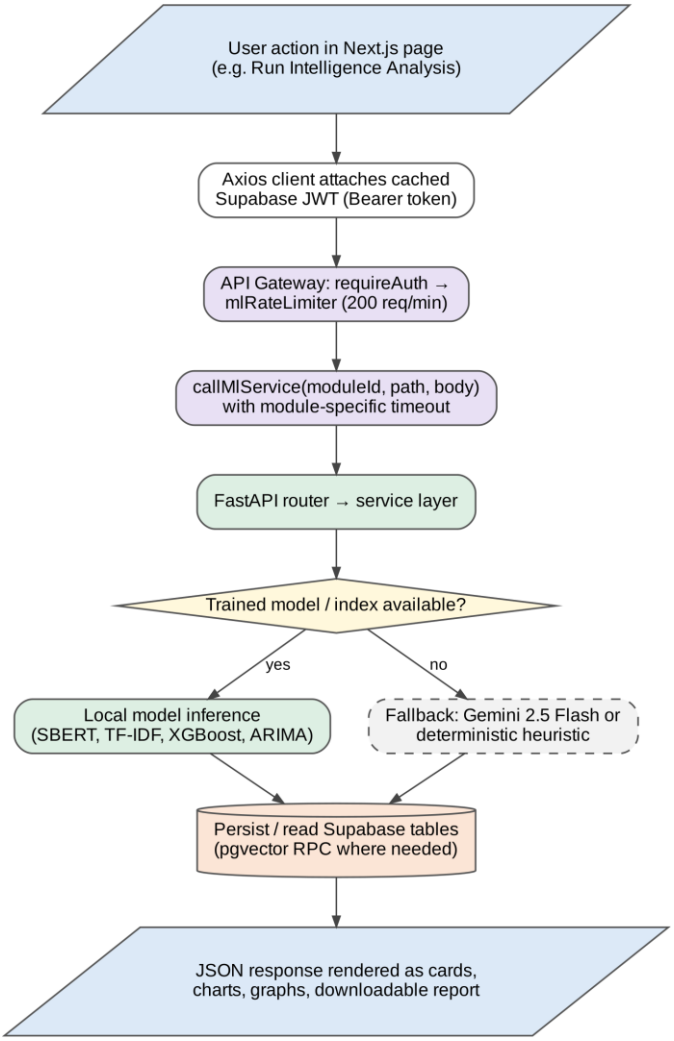


Figure 5.7: Request flow from a Next.js page through the gateway to a FastAPI service, including the fallback decision

5.4 Verification and Software Testing

Testing was performed at three levels. **Unit-level checks** exercised the citation engine directly on the benchmark and style test set (Section 6.1). **Component-level functional validation** executed 13 test cases against the running FastAPI service on 28 August 2026; Table 5.3 lists them with their verdicts. **End-to-end checks** used the Playwright specification `e2e/module1-citation.spec.ts` and manual walkthroughs of every page through the gateway.

Table 5.3: Functional validation of Module 1 (28 August 2026)

ID	Feature / scenario	Result	Evidence or root cause
M1-01	Parser — APA journal reference	PARTIAL	Authors, title, year correct; first character of journal dropped ("ournal…"); volume/issue left inside venue
M1-02	Parser — IEEE-formatted reference	FAIL → fixed	Authors empty, title wrong; fixed on 29 Aug by quoted-title anchoring (re-tested: 3 authors, title, year correct)
M1-03	Parser — unstructured text	PASS	No crash; warnings for missing author, year and journal
M1-04	Formatter — structured record (IEEE)	PASS	Exact IEEE output with vol., no., pp. and DOI
M1-05	DOI lookup — 10.1038/nphys1170	PASS	Correct CrossRef metadata (Aspelmeyer, Nature Physics, 2009)
M1-06	Similar papers — plant-disease query	PASS	All five SLIIT papers on topic
M1-07	Reference list — APA ordering	PASS	Sorted by first-author surname
M1-08	Gap analysis — specific compound topic	PARTIAL	Pipeline correct; several gaps only loosely related (similarity 0.31–0.41)
M1-09	Gap analysis — out-of-corpus topic	PARTIAL	No confidence gate: six weak gaps (0.15–0.33) returned
M1-10	Gap index status	PASS	553 gaps, 494 papers, 384-d
M1-11	Proposal — IoT smart irrigation	PASS	All five exemplars on topic; sections traceable
M1-12	Proposal index status	PASS	3,858 exemplars, 384-d
M1-15	Mind map — CNN plant-disease abstract	PARTIAL	Gemini JSON parse failed; fallback used ("investigates" as centre)

5.5 Deployment and Operation

The service is built from `services/module1-integrity/Dockerfile` with the SBERT model and indices baked into the image to avoid a cold-start download; Railway rebuilds it on every push to `main` through its GitHub integration, while GitHub Actions deploys the front end to Vercel. Environment variables (`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `GEMINI_API_KEY`) are injected at runtime and never committed. Each index exposes a status endpoint (`/gaps/status`, `/proposals/status`) used by the dashboard's model-status grid. Figure 5.8 shows the deployment pipeline.

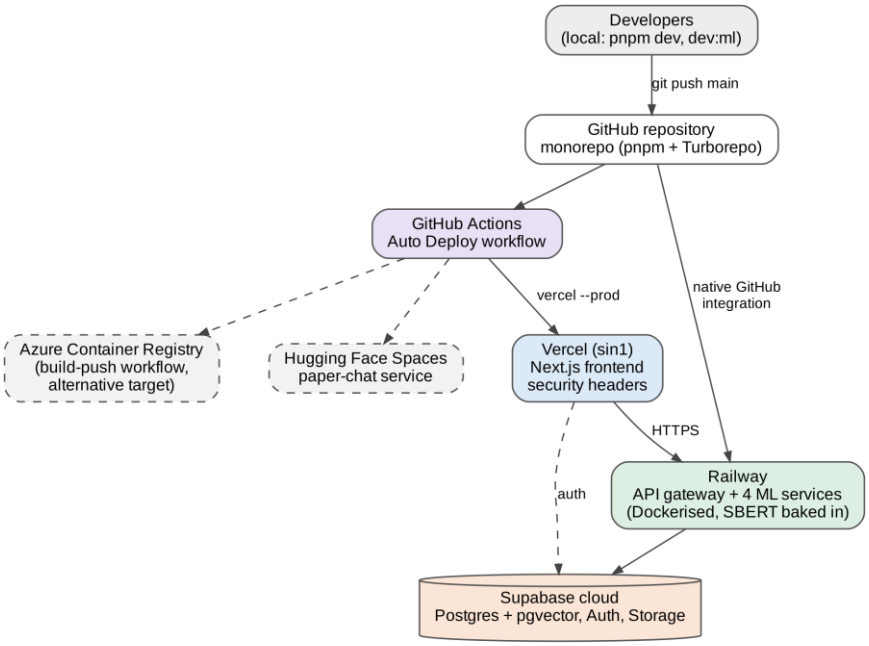


Figure 5.8: Continuous deployment pipeline of the platform

6 RESULTS AND VALIDATION

6.1 Research Results

6.1.1 RQ1: Citation parsing accuracy

The current citation engine was run over 3,866 references built from corpus metadata (*Authors (Year). Title.* shape). Figure 6.1 and Table 6.1 report field-level accuracy. Year extraction was perfect, titles were exact in 98.0% of cases and the number of authors in 96.5%; author surnames matched in order in 90.9%, and all three fields were simultaneously correct in 89.1%. Median parse time was 0.077 ms (95th percentile 0.127 ms) on a single CPU core.

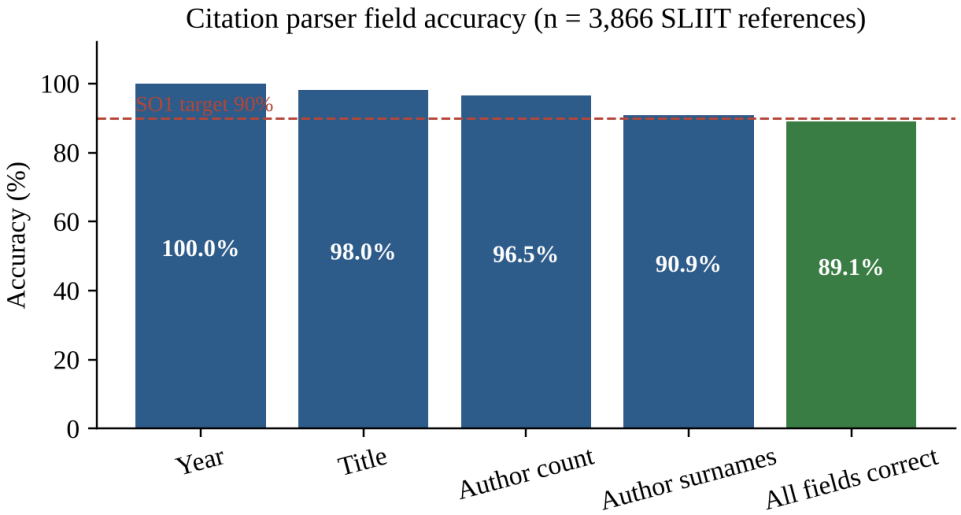


Figure 6.1: Citation parser field-level accuracy on 3,866 SLIIT references

Table 6.1: Citation parser benchmark (n = 3,866)

Field	Exact-match accuracy	Target (SO1)	Met?
Publication year	100.0%	≥ 90%	Yes
Title	98.0%	≥ 90%	Yes
Number of authors	96.5%	≥ 90%	Yes
Author surnames (ordered)	90.9%	≥ 90%	Yes
All fields simultaneously	89.1%	—	—
Median latency	0.077 ms	< 100 ms	Yes

The six-style robustness set showed that, after the 29 August fix, APA journal, IEEE conference, IEEE journal, book and website references all yield correct authors, titles,

years and source types, and the unstructured string produces three appropriate warnings. Two defects remain: (i) the first character of a journal name is dropped when the venue follows a sentence-final full stop (APA journal shape), and (ii) APA-style volume/issue such as "12(3)" is left inside the venue rather than parsed, and IEEE conference venues retain a leading "in", producing "in *in Proc.*" in IEEE output. Both are localised regex issues (Section 7.5).

For comparison, the spaCy transformer NER model trained during the project on 3,290 synthetic references reached a best validation F1 of 0.995 on 581 held-out references. Because training and validation strings were produced by the same template, this figure measures template learning rather than real-world robustness, which is one reason the deterministic engine was chosen for production.

6.1.2 RQ2: Gap retrieval relevance

For a well-covered topic, retrieval returned six ranked gaps with similarity 0.31–0.41 and novelty 43–65%, all grounded in real SLIIT papers (Figure 5.4). The strongest gaps were domain/contextual gaps on Sri Lankan agricultural applications. However, validation cases M1-08 and M1-09 showed that, because the index contains only 553 gap sentences (Figure 6.2), loosely related gaps are also returned, and for an out-of-corpus topic (quantum cryptography for underwater acoustic networks) the system still returned six gaps with similarity as low as 0.15, presented with the same confidence as strong matches. RQ2 is therefore answered positively for topics represented in the corpus and negatively for poorly covered topics, which motivates a confidence gate (Section 9.4).

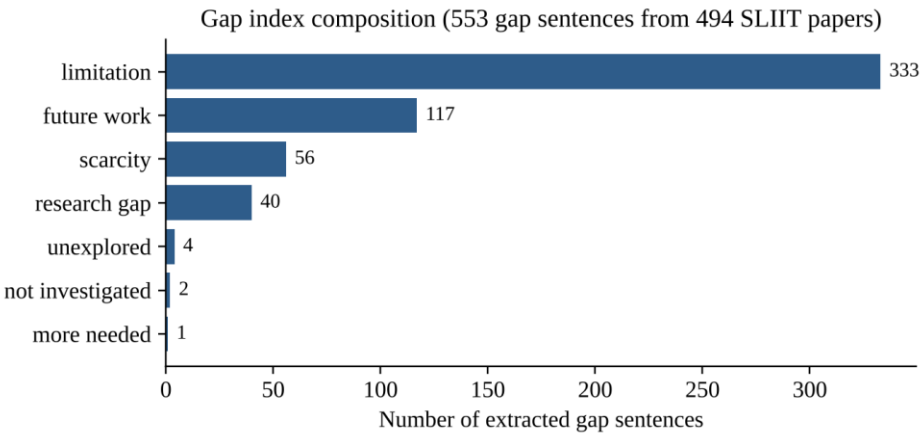


Figure 6.2: Composition of the gap index by gap type

6.1.3 RQ3: Grounded proposal drafting

For "IoT-based smart irrigation for paddy fields" (domain: agriculture technology) the generator retrieved five on-topic exemplars with similarity 0.63–0.72 — an automated water-gate controller for paddy fields, IoT crop-yield forecasting, IoT crop recommendation, smart gardening irrigation and domestic smart irrigation — and every section of the draft cites or paraphrases one of them (Figures 5.5 and 5.6). Because composition is a CPU matrix product followed by string operations, the draft is produced without GPU inference. The trade-off is fluency: the methodology paragraph concatenates snippets and can read unevenly, which is why the interface presents the draft as a starting point to be rewritten.

6.2 Product and Technical Results

Table 6.2: Feature completion status of Module 1

Feature	Status	Notes
Citation parser / formatter / reference list	Complete	Deterministic; two known regex defects
DOI and title lookup	Complete	CrossRef; verified against real DOI
Similar SLIIT papers	Complete	3,858-paper SBERT index
Gap analysis (topic, text, PDF) + HTML report	Complete	No confidence gate yet
Proposal generator + PDF export	Complete	Retrieval-first, Gemini fallback
Mind map builder + SVG/PNG export	Complete	Fallback lacks a source indicator
Citation NER and LoRA proposal model	Prototype	Wrapper classes present; not routed

Non-functional evidence: citation parsing is four orders of magnitude below the 100 ms target; retrieval endpoints load pre-computed indices once and score with a single matrix product; every route sits behind the gateway's JWT and rate-limit middleware; and fallback tiers were exercised in validation (M1-15), where the endpoint returned a usable result rather than an error even though Gemini failed.

6.3 User and Stakeholder Validation

The module validation report, including all 13 test cases and their evidence, was reviewed by the supervisor, Ms. Sanjeevi Chandrasiri, who signed it off with the comment that the output had been reviewed, validated and confirmed. Her review emphasised the strength of the retrieval-based features (similar papers and proposal generation) and the need to address the citation and gap-confidence findings before production use. A structured user-acceptance protocol for student users is provided in Appendix D; its results are to be added after the final presentation sessions.

6.4 Integration Results

All 15 Module 1 endpoints are reachable through the gateway under `/api/v1/`, with JWT authentication and rate limiting applied, and the four pages render within the shared dashboard shell. The SBERT model shared with Module 3 loads once per service and is reused by gap analysis, proposal retrieval and similar-paper search.

6.5 Summary of Findings

The deterministic citation engine meets the 90% field-level target on 3,866 references with sub-millisecond latency; retrieval-grounded proposal generation produces traceable drafts; and gap retrieval works well for covered topics but needs a confidence gate.

Overall, 8 of 13 validation cases passed, 4 were partial and 1 failed (since fixed) (Figure 6.3).

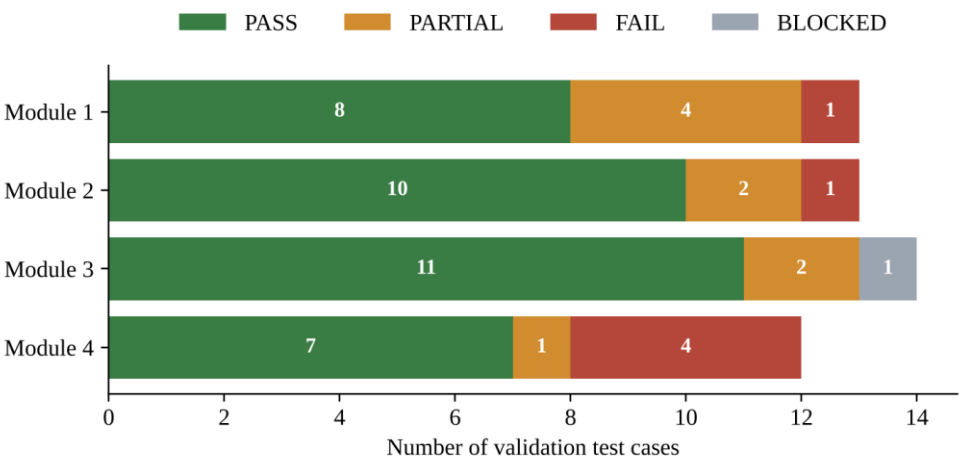


Figure 6.3: Validation outcomes per module (Module 1 in the top row)

7 DISCUSSION

7.1 Interpretation of Findings

The results support the design principle stated in Section 2.5. Where rules exist, a deterministic engine reaches high accuracy at negligible cost and, crucially for an integrity tool, behaves identically every time: a student and a supervisor will see the same citation. The main failure (M1-02) was a *shape* problem — IEEE strings put the year at the end — and was fixed by anchoring on the quoted title rather than by adding a model. Where evidence exists, retrieval provides something an LLM cannot: a citable source for every claim. The gap results show the boundary of that approach: retrieval quality is bounded by index coverage, and a small index (553 gap sentences) must be paired with an explicit "no confident match" response.

7.2 Comparison with Literature Baselines and Existing Solutions

Published reference parsers such as CERMINE and GROBID report high field accuracy on bibliographies extracted from PDFs [8], [9]; the present engine reaches comparable title and year accuracy on SLIIT-style strings without training data but is less robust to unusual layouts, and unlike those tools it renders submission-ready APA and IEEE text. Compared with LLM idea generators [11], [12], the gap module is less creative but every gap is a sentence written by a real author in a real paper. No head-to-head user study against these products was conducted, so comparisons are functional rather than experimental.

7.3 Research Contribution

- A retrieval-first architecture for academic-integrity support that combines deterministic citation formatting, evidence-backed gap retrieval and grounded proposal composition, with explicit fallback tiers.
- A gap index built from author-stated limitations and future-work sentences in an institutional repository, with a transparent multi-dimensional ranking (similarity, type, recency, novelty) and de-duplication.

- An empirical benchmark of rule-based citation parsing on 3,866 institutional references, including the observation that template-trained NER scores are not a reliable indicator of real-world robustness.

7.4 Product Contribution

Students gain one place in which to format references, justify a topic, draft a proposal and map its concepts, with every output linked to SLIIT research they can cite. Supervisors gain proposals that arrive already grounded in prior work. The department gains a tool that reflects its own research history rather than generic web content.

7.5 Limitations and Threats to Validity

- **Benchmark validity:** the 3,866 references share one template generated from metadata; real student references are more varied, so the benchmark overestimates accuracy on messy input. The six-style set mitigates but does not remove this threat.
- **Open defects:** dropped first character of APA journal names, unparsed "vol(issue)" notation and a duplicated "in" for IEEE conference venues.
- **Gap coverage:** 553 gap sentences cover only a fraction of topics; there is no confidence gate.
- **SBERT fine-tuning:** the "SLIIT-tuned" encoder was trained on only nine triplets, so it behaves essentially like the base model; the claimed domain adaptation is minimal.
- **Generative dependence:** mind maps rely on Gemini; JSON-parse failures fall back silently.
- **User evidence:** expert sign-off was obtained, but a student user study is still pending.

7.6 Implications

For practice, the work shows that a useful integrity assistant can run on CPU-only infrastructure if retrieval and rules do most of the work. For research, it suggests evaluating gap-identification systems on *evidence quality* — whether a gap is attested

in the literature — alongside novelty. For the institution, it highlights the value of maintaining a clean, embedded repository: every Module 1 feature improves as the corpus grows.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

Primary users are undergraduate and postgraduate research students (approximately 20–28 years old) in Sri Lankan and South Asian universities who need support with referencing, topic justification and proposal writing. **Secondary users** are supervisors and research coordinators who review proposals. The value proposition is a single tool that replaces a reference manager, a search engine and a mind-mapping application for the proposal stage, with outputs grounded in the institution's own research.

8.2 Market and Competitive Position

Existing tools each cover one task (Table 2.2). General-purpose LLM assistants cover several tasks but cannot guarantee correct citations or verifiable gaps. Module 1 differentiates on three points: institution-specific evidence, deterministic citation output and pre-submission semantic screening. Because it is part of an integrated platform, it also benefits from network effects inside a university — the more papers a department deposits, the better every feature becomes.

8.3 Business and Sustainability Model

Table 8.1: Indicative revenue model (from the approved proposal)

Phase	Strategy	Indicative pricing
Year 1	Free for SLIIT students and staff; beta testing and refinement	—
Year 2	Freemium: citation tools free; gap analysis and proposal drafting premium	LKR 500 per student per month
Year 3	Institutional licensing with corpus onboarding and support	LKR 150,000 per institution per year

Running costs are modest because inference runs on CPU: the proposal budget estimated LKR 120,000 per year for cloud hosting and LKR 60,000 per year for LLM API use, within a development budget of LKR 115,000. Gemini is used only as a fallback, which keeps API costs proportional to failures rather than to traffic.

8.4 Scalability, Intellectual Property and Compliance

- The service is stateless and containerised; multiple replicas can sit behind the gateway. Exact search over a 3,858 × 384 matrix scales linearly; beyond roughly 10^5 papers an HNSW index in pgvector [23], [24] would replace it.
- The gap-ranking formula, the three-tier fallback design and the composition templates are the team's original work; third-party libraries are used under their open-source licences.
- Corpus records are public metadata; uploaded drafts are private to the user. Compliance with the PDPA [32] and the institution's academic-integrity policy is required before any external deployment.

8.5 Contribution to the Sustainable Development Goals

The platform contributes primarily to **SDG 4 (Quality Education)**, in particular target 4.3 (equal access to quality tertiary education) and target 4.4 (relevant skills), by giving every student — not only those with well-connected supervisors or paid tools — structured support for research. It also contributes to **SDG 9 (Industry, Innovation and Infrastructure)**, target 9.5 (enhance scientific research and upgrade technological capabilities), by turning an institutional repository into queryable research infrastructure [44]. Module 1 contributes specifically by lowering the skill barrier to correct referencing and proposal writing, which disproportionately benefits students without prior research experience, and by promoting academic integrity through early, explainable feedback.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the Research Integrity and Compliance Module of the Researchly AI platform. It showed that a retrieval-first, rules-where-possible design can support the writing stage of student research on modest infrastructure: a deterministic citation engine met the 90% field-level target with sub-millisecond latency, proposal drafts were grounded in identifiable SLIIT papers, and gap retrieval surfaced evidenced gaps for well-covered topics. Validation also exposed concrete weaknesses — citation-shape defects, and the absence of a confidence gate for gaps — that are documented with root causes.

9.2 Achievement of Objectives

Table 9.1: Achievement of objectives

Objective	Status	Evidence
SO1 Citation parser and APA/IEEE formatter (≥ 90%)	Achieved (with known defects)	Section 6.1.1; Table 6.1
SO2 Gap identification returning 3–5 evidenced gaps	Partly achieved	Section 6.1.2; M1-08, M1-09
SO3 Structured, grounded proposal assistant	Achieved (retrieval instead of fine-tuned LLM)	Section 6.1.3; M1-11
SO4 Mind-map generator	Achieved (best-effort)	Section 4.4.4; M1-15
SO5 Integration into the platform	Achieved	Section 6.4

9.3 Individual Research and Product Contributions

- Designed and implemented all five Module 1 features, their APIs and user interfaces.
- Built the gap and proposal indices and the multi-dimensional gap-ranking and intelligence layer.
- Benchmarked the citation engine on 3,866 references and fixed the IEEE-shape defect identified in validation.
- Built the shared integration infrastructure (monorepo, API gateway, CI/CD, Vercel/Railway deployment).

9.4 Future Directions

- Add a confidence gate to gap analysis (for example, return "no strong gaps found" when the best similarity is below 0.35) and expand the gap index to all 3,858 abstracts.
- Fix the remaining citation regex defects and add BibTeX/RIS import and export.
- Fine-tune the SBERT encoder on a realistic set of SLIIT query–paper pairs rather than nine triplets.
- Expose a `source` field (gemini or fallback) in mind-map responses and evaluate a KeyBERT-based local alternative [36].
- Conduct the student user-acceptance study in Appendix D and measure drafting-time savings.

REFERENCES

[1] National Science Board, "Publications output: U.S. trends and international comparisons," Science and Engineering Indicators 2024, NSB-2023-33, National Science Foundation, Alexandria, VA, USA, 2023.

[2] T. Foltýnek, N. Meuschke, and B. Gipp, "Academic plagiarism detection: A systematic literature review," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–42, 2019.

[3] J. Beel, B. Gipp, S. Langer, and C. Breitinger, "Research-paper recommender systems: A literature survey," *Int. J. Digit. Libraries*, vol. 17, no. 4, pp. 305–338, 2016.

[4] American Psychological Association, *Publication Manual of the American Psychological Association*, 7th ed. Washington, DC, USA: APA, 2020.

[5] IEEE, "IEEE reference guide," IEEE Periodicals, Piscataway, NJ, USA, 2023. [Online]. Available: https://journals.ieeeauthorcenter.ieee.org/

[6] Crossref, "REST API documentation," Crossref, 2024. [Online]. Available: https://www.crossref.org/documentation/retrieve-metadata/rest-api/

[7] Corporation for Digital Scholarship, "Zotero," 2024. [Online]. Available: https://www.zotero.org/

[8] D. Tkaczyk, P. Szostek, M. Fedoryszak, P. J. Dendek, and Ł. Bolikowski, "CERMINE: Automatic extraction of structured metadata from scientific literature," *Int. J. Document Anal. Recognit.*, vol. 18, no. 4, pp. 317–335, 2015.

[9] P. Lopez, "GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications," in *Proc. Eur. Conf. Digit. Libraries (ECDL)*, Corfu, Greece, 2009, pp. 473–474.

[10] I. G. Councill, C. L. Giles, and M.-Y. Kan, "ParsCit: An open-source CRF reference string parsing package," in *Proc. 6th Int. Conf. Lang. Resour. Eval. (LREC)*, Marrakech, Morocco, 2008, pp. 661–667.

[11] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang, "ResearchAgent: Iterative research idea generation over scientific literature with large language models," in *Proc. NAACL-HLT*, Albuquerque, NM, USA, 2025.

[12] C. Si, D. Yang, and T. Hashimoto, "Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers," arXiv:2409.04109, 2024.

[13] S. Iqbal, S.-U. Hassan, N. R. Aljohani, S. Alelyani, R. Nawaz, and L. Bornmann, "A decade of in-text citation analysis based on natural language processing and machine learning techniques: An overview of empirical studies," *Scientometrics*, vol. 126, no. 8, pp. 6551–6599, 2021.

[14] R. Jha, A.-A. Jbara, V. Qazvinian, and D. R. Radev, "NLP-driven citation analysis for scientometrics," *Natural Lang. Eng.*, vol. 23, no. 1, pp. 93–130, 2017.

[15] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[16] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 9459–9474.

[17] Y. Wei, H. Guo, J. Wei, and Z. Su, "Revealing semantic structures of texts: Multi-grained framework for automatic mind-map generation," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, Macao, China, 2019.

[18] Y. Wen, Z. Wang, and J. Sun, "MindMap: Knowledge graph prompting sparks graph of thoughts in large language models," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Bangkok, Thailand, 2024.

[19] Sri Lanka Institute of Information Technology, "Guidelines on documentation and submission of dissertations," Faculty of Computing, SLIIT, Malabe, Sri Lanka, 2025.

[20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[21] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP-IJCNLP)*, Hong Kong, 2019, pp. 3982–3992.

[22] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 5776–5788.

[23] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[24] A. Kane, "pgvector: Open-source vector similarity search for Postgres," GitHub repository, 2024. [Online]. Available: https://github.com/pgvector/pgvector

[25] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[26] T. Buzan, *Mind Map Mastery: The Complete Guide to Learning and Using the Most Powerful Thinking Tool in the Universe*. London, U.K.: Watkins, 2018.

[27] J. D. Novak and A. J. Cañas, "The theory underlying concept maps and how to construct and use them," Florida Inst. Human Mach. Cognition, Pensacola, FL, USA, Tech. Rep. IHMC CmapTools 2006-01 Rev 01-2008, 2008.

[28] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[29] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[30] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *J. Manage. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007.

[31] Supabase, "Row Level Security," Supabase Documentation, 2024. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security

[32] Parliament of the Democratic Socialist Republic of Sri Lanka, *Personal Data Protection Act, No. 9 of 2022*. Colombo, Sri Lanka: Government Publications Bureau, 2022.

[33] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[34] M. Grootendorst, "BERTopic: Neural topic modeling with a class-based TF-IDF procedure," arXiv:2203.05794, 2022.

[35] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," *J. Mach. Learn. Res.*, vol. 3, pp. 993–1022, 2003.

[36] M. Grootendorst, "KeyBERT: Minimal keyword extraction with BERT," Zenodo, 2020, doi: 10.5281/zenodo.4461265.

[37] ISO/IEC 25010:2011, *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.

[38] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[39] Vercel, "Next.js documentation: App Router," 2024. [Online]. Available: https://nextjs.org/docs/app

[40] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning, 2018.

[41] S. Ramírez, "FastAPI," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[42] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[43] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, Art. no. 2, 2014.

[44] United Nations, *The Sustainable Development Goals Report 2023: Special Edition*. New York, NY, USA: United Nations, 2023.

[45] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

GLOSSARY

Term	Definition
Gap statement	A sentence in which authors state a limitation, open problem or future work in their own paper.
Exemplar	A prior SLIIT paper retrieved as the basis for a generated proposal section.
Fallback tier	A simpler method used automatically when the primary model or service is unavailable.
Confidence (citation)	A weighted 0–1 score of how many bibliographic fields were extracted.
Sentence embedding	A fixed-length vector representing the meaning of a sentence, compared by cosine similarity.
Saturated topic	A term occurring in four or more matched paper titles, indicating a well-explored area.

APPENDICES

Appendix A: Extended Results and Test Evidence

Table A.1 lists the six-style robustness test executed on the final version of the citation engine.

Table A.1: Citation engine behaviour on six reference shapes (final version)

Shape	Detected type	Authors	Year	Warnings
APA journal	journal	2	2021	none
IEEE conference	conference	3	2020	none
IEEE journal (ACM Comput. Surv.)	journal	3	2019	none
Book (Box et al.)	book	4	2015	none
Website (WHO)	website	1	2023	none
Unstructured string	journal	0	—	author, year, journal missing

Recorded request for validation case M1-11 (proposal generation):

```
POST /api/v1/proposals/generate
{ "topic": "IoT-based smart irrigation for paddy fields",
  "domain": "Agriculture Technology", "top_k": 5 }
```

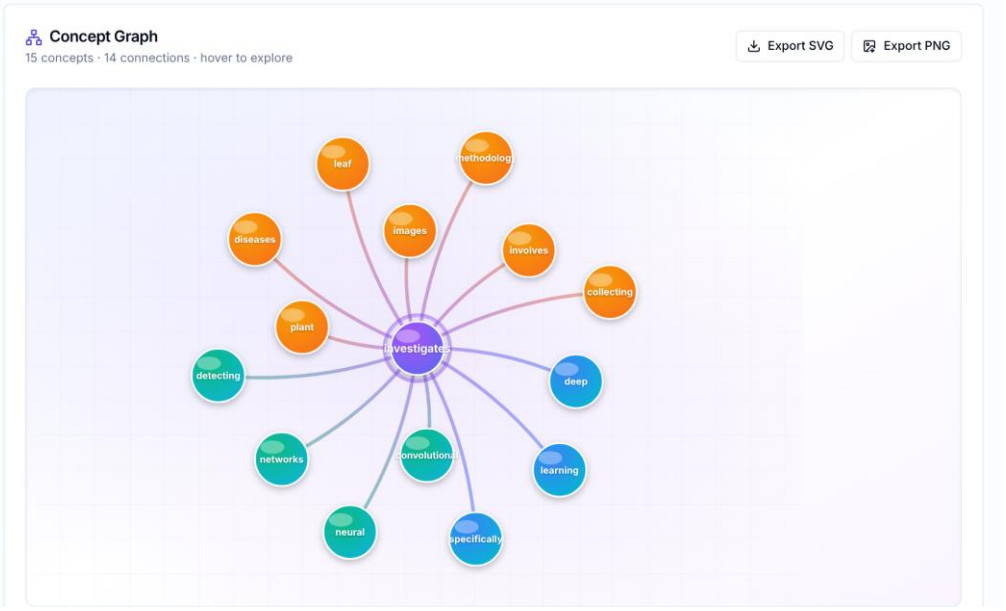


Figure A.1: Mind Map Builder output for a CNN plant-disease abstract, produced by the local fallback when Gemini JSON parsing failed (validation case M1-15)

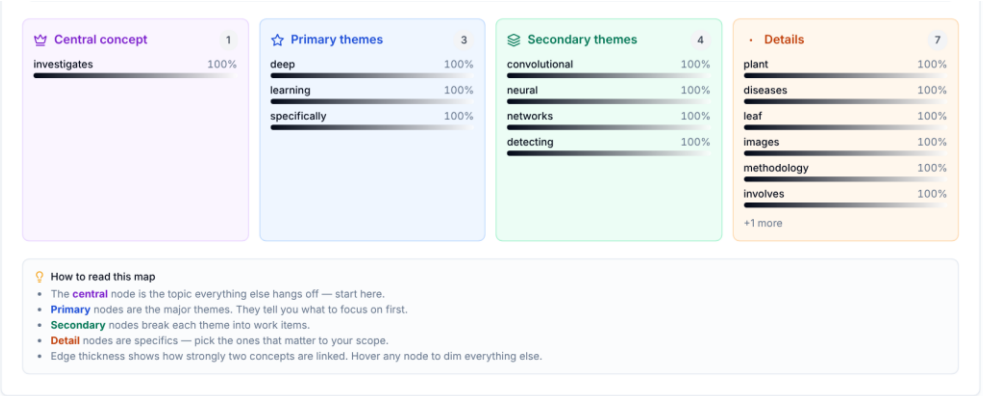


Figure A.2: Tiered concept summary (central, primary, secondary, detail) under the mind-map graph

Appendix B: Academic Integrity and AI-Use Disclosure

In accordance with the IT4010 policy, the following AI assistance is disclosed. The author verified all content against the repository, the validation evidence and the cited sources, and remains responsible for it.

Table B.1: AI-use disclosure

Tool	Purpose	Verification performed	Sections affected
Claude (Anthropic) — AI coding assistant	Debugging suggestions; automated execution of the module validation harness and drafting of the module reports (28 Aug 2026)	Each test was re-run and its evidence inspected; supervisor sign-off obtained	Chapter 5.4, Chapter 6
Claude (Anthropic)	Drafting and formatting of this thesis from repository evidence; generation of diagrams and charts from repository data	All figures recomputed from repository files; every number cross-checked with source JSON/CSV; references checked	All chapters
Google Gemini 2.5 Flash	Runtime component (mind maps, proposal fallback)	Outputs treated as untrusted; fallbacks implemented	Chapter 4

Appendix C: Additional Interface Screenshots

Screenshots in this thesis were captured from the production build of the Researchly AI front end (Next.js) rendering the responses recorded from the live Module 1 service during the 28 August 2026 validation run; the citation parser screenshots use the current version of the citation engine.



Figure C.1: Cross-domain opportunities produced by the gap intelligence layer

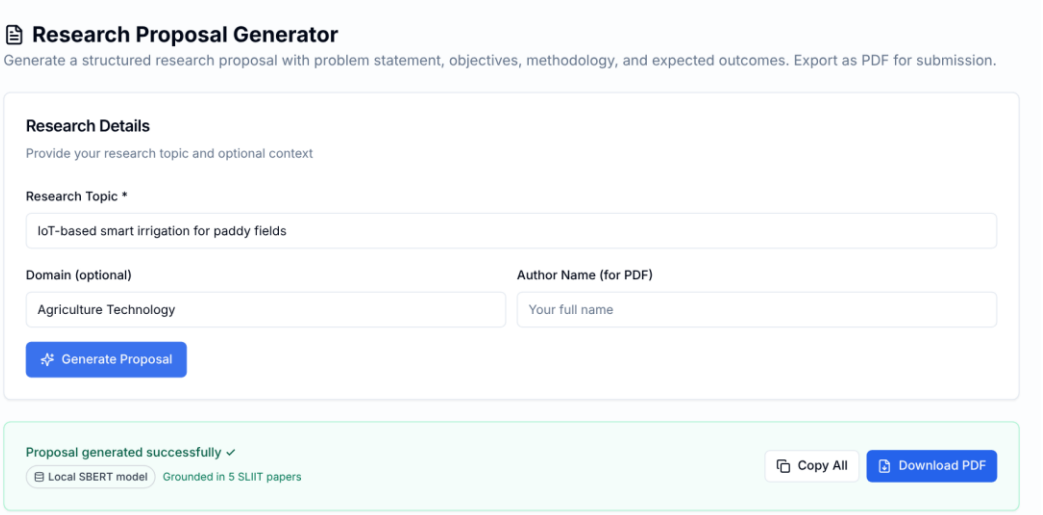


Figure C.2: Proposal generator input form with topic and domain

Appendix D: User Acceptance Testing Protocol

The following task-based protocol, aligned with the IT4010 guidance that users should perform realistic tasks, is used for student user-acceptance testing. Each participant completes the tasks without assistance; completion, time and errors are observed, followed by the System Usability Scale [45].

Table D.1: UAT tasks and recording sheet for Module 1

Task	Expected outcome	Completed (Y/N)	Time (s)	Errors / comments
T1 Paste three of your own references and copy them in IEEE style	Correct IEEE entries			
T2 Look up a paper by DOI and add it to the reference list	Entry added in order			
T3 Run gap analysis on your project topic and download the report	HTML report saved			
T4 Generate a proposal for your topic and identify its source papers	Sources named			
T5 Build a mind map from your abstract and export PNG	PNG exported			